Компьютерная графика

Лекция 8: Геометрические шейдеры, compute шейдеры, storage buffers, shader atomics, shadow volumes, shadow mapping

---

2026

Виды шейдеров

- Render pipeline
  - Vertex shader
  - Tessellation control shader
  - Tessellation evaluation shader
  - Geometry shader
  - Fragment shader

Виды шейдеров

- Render pipeline
  - Vertex shader
  - Tessellation control shader
  - Tessellation evaluation shader
  - Geometry shader
  - Fragment shader
- Mesh shading pipeline
  - Task shader
  - Mesh shader
  - Fragment shader

Виды шейдеров

- Render pipeline
  - Vertex shader
  - Tessellation control shader
  - Tessellation evaluation shader
  - Geometry shader
  - Fragment shader
- Mesh shading pipeline
  - Task shader
  - Mesh shader
  - Fragment shader
- Ray tracing pipeline
  - Ray generation shader
  - Intersection shader
  - Any-hit shader
  - Closest-hit shader
  - Miss shader

Виды шейдеров

- Render pipeline
  - Vertex shader
  - Tessellation control shader
  - Tessellation evaluation shader
  - Geometry shader
  - Fragment shader
- Mesh shading pipeline
  - Task shader
  - Mesh shader
  - Fragment shader
- Ray tracing pipeline
  - Ray generation shader
  - Intersection shader
  - Any-hit shader
  - Closest-hit shader
  - Miss shader
- Compute pipeline
  - Compute shader

Виды шейдеров

- Из-за кросс-платформенности, WebGPU поддерживает только базовый **render pipeline (vertex + fragment)** и **compute pipeline**

Виды шейдеров

- Из-за кросс-платформенности, WebGPU поддерживает только базовый **render pipeline (vertex + fragment)** и **compute pipeline**
- В wgpu есть экспериментальная поддержка **mesh shaders** и **ray queries**, ведётся работа над поддержкой **ray tracing pipelines**

Геометрические шейдеры

- Опциональный шейдер, встраивающийся после вершинного шейдера, до perspective divide

Геометрические шейдеры

- Опциональный шейдер, встраивающийся после вершинного шейдера, до perspective divide
- Оперирует целыми примитивами (точками/линиями/треугольниками), т.е. наборами вершин

Геометрические шейдеры

- Опциональный шейдер, встраивающийся после вершинного шейдера, до perspective divide
- Оперирует целыми примитивами (точками/линиями/треугольниками), т.е. наборами вершин
- Может на лету менять тип примитива и количество вершин

Геометрические шейдеры

- Опциональный шейдер, встраивающийся после вершинного шейдера, до perspective divide
- Оперирует целыми примитивами (точками/линиями/треугольниками), т.е. наборами вершин
- Может на лету менять тип примитива и количество вершин
- Поддерживаются в OpenGL, DirectX, Vulkan

Геометрические шейдеры

- Опциональный шейдер, встраивающийся после вершинного шейдера, до perspective divide
- Оперирует целыми примитивами (точками/линиями/треугольниками), т.е. наборами вершин
- Может на лету менять тип примитива и количество вершин
- Поддерживаются в OpenGL, DirectX, Vulkan
- **Не поддерживается** в WebGPU, Metal

Геометрические шейдеры

- Геометрические шейдеры тяжело ложатся на архитектуру GPU из-за отсутствия чёткого соответствия входов и выходов

Геометрические шейдеры

- Геометрические шейдеры тяжело ложатся на архитектуру GPU из-за отсутствия чёткого соответствия входов и выходов
- Требуют дополнительной временной памяти, не используют вершинный кеш

Геометрические шейдеры

- Геометрические шейдеры тяжело ложатся на архитектуру GPU из-за отсутствия чёткого соответствия входов и выходов
- Требуют дополнительной временной памяти, не используют вершинный кеш
- Производительность очень сильно зависит от GPU, драйвера, и самого шейдера

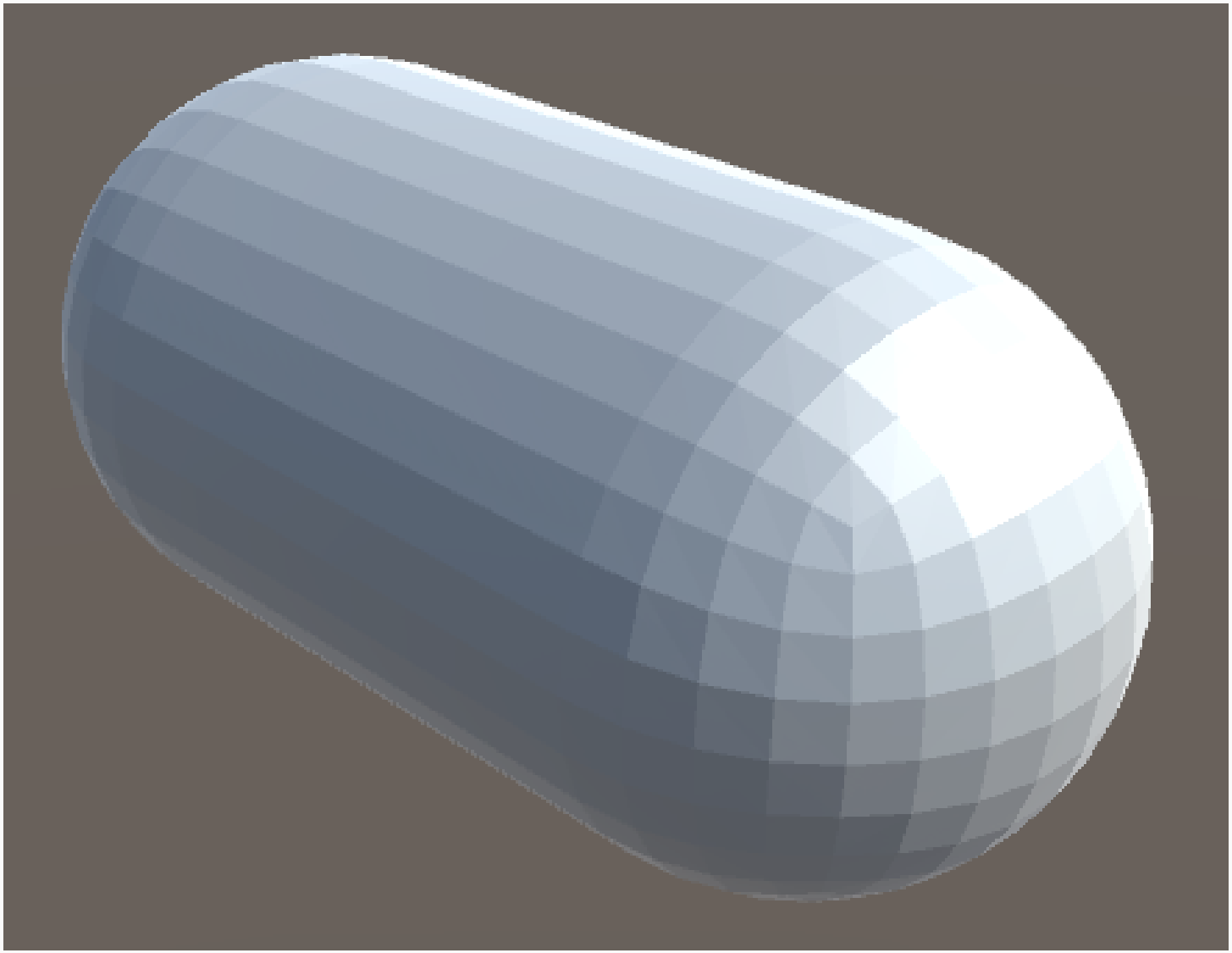
Геометрические шейдеры

- Геометрические шейдеры тяжело ложатся на архитектуру GPU из-за отсутствия чёткого соответствия входов и выходов
- Требуют дополнительной временной памяти, не используют вершинный кеш
- Производительность очень сильно зависит от GPU, драйвера, и самого шейдера
- В современности их стараются не использовать, заменяя более надёжными инструментами: более сложными вершинными шейдерами, instancing'ом, compute шейдерами

Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading’a

Flat shading



Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading’a

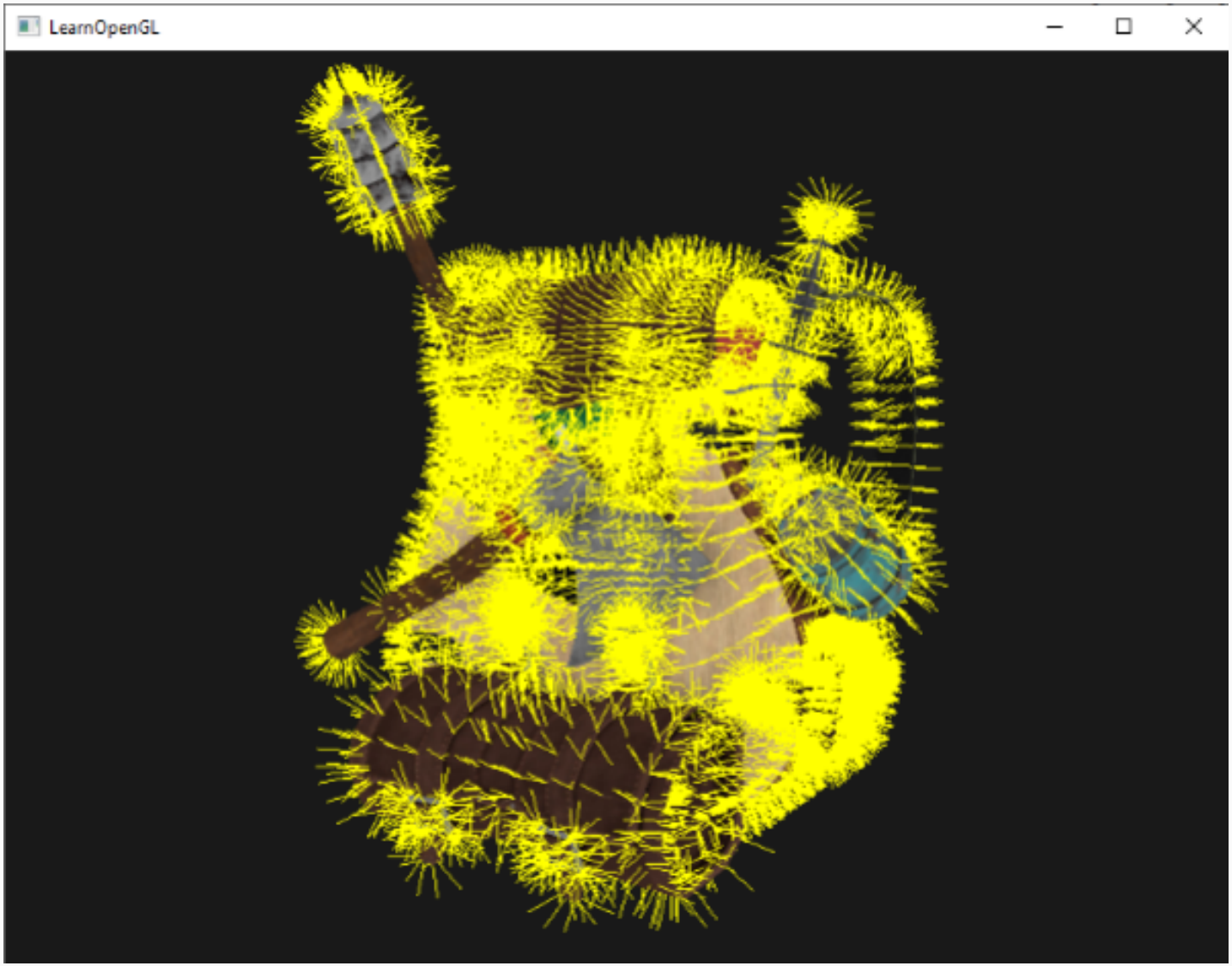
Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading’a
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью

Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading'a
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью
- Визуализация нормалей

Визуализация нормалей



Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading’a
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью
- Визуализация нормалей

Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading'а
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью
- Визуализация нормалей
  - Вход: вершина с нормалью
  - Выход: линия из двух вершин – исходная вершина, исходная вершина + нормаль

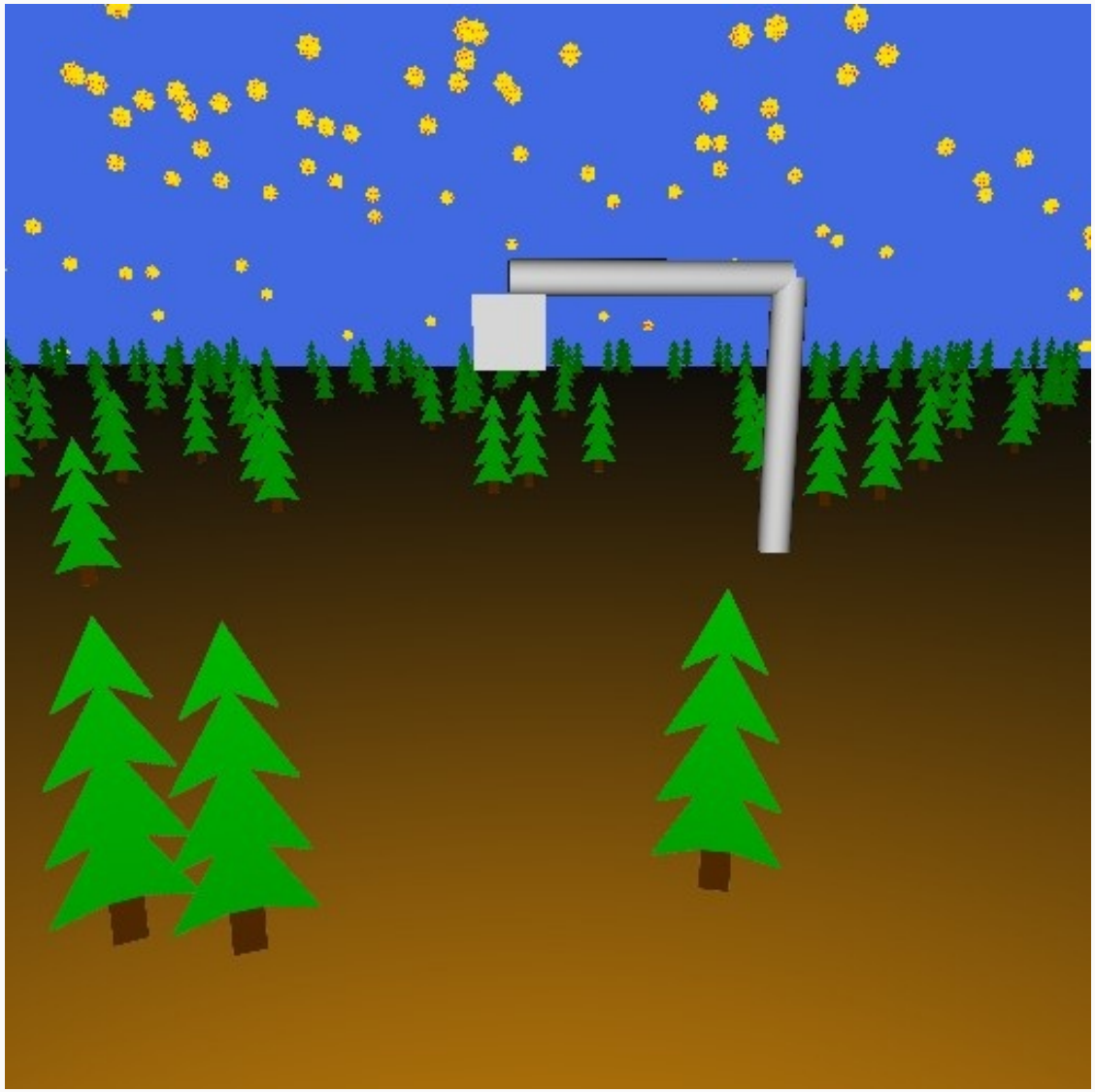
Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading'а
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью
- Визуализация нормалей
  - Вход: вершина с нормалью
  - Выход: линия из двух вершин – исходная вершина, исходная вершина + нормаль
- Shadow volumes – алгоритм рисования теней

Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading'a
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью
- Визуализация нормалей
  - Вход: вершина с нормалью
  - Выход: линия из двух вершин – исходная вершина, исходная вершина + нормаль
- Shadow volumes – алгоритм рисования теней
- Billboards – плоские фигуры, всегда смотрящие в сторону камеры

Billboards



Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading'a
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью
- Визуализация нормалей
  - Вход: вершина с нормалью
  - Выход: линия из двух вершин – исходная вершина, исходная вершина + нормаль
- Shadow volumes – алгоритм рисования теней
- Billboards – плоские фигуры, всегда смотрящие в сторону камеры

Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading'a
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью
- Визуализация нормалей
  - Вход: вершина с нормалью
  - Выход: линия из двух вершин – исходная вершина, исходная вершина + нормаль
- Shadow volumes – алгоритм рисования теней
- Billboards – плоские фигуры, всегда смотрящие в сторону камеры
  - Вход: одна вершина (точка)
  - Выход: набор треугольников

Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading'a
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью
- Визуализация нормалей
  - Вход: вершина с нормалью
  - Выход: линия из двух вершин – исходная вершина, исходная вершина + нормаль
- Shadow volumes – алгоритм рисования теней
- Billboards – плоские фигуры, всегда смотрящие в сторону камеры
  - Вход: одна вершина (точка)
  - Выход: набор треугольников
  - Системы частиц

Billboards: система частиц (дым)



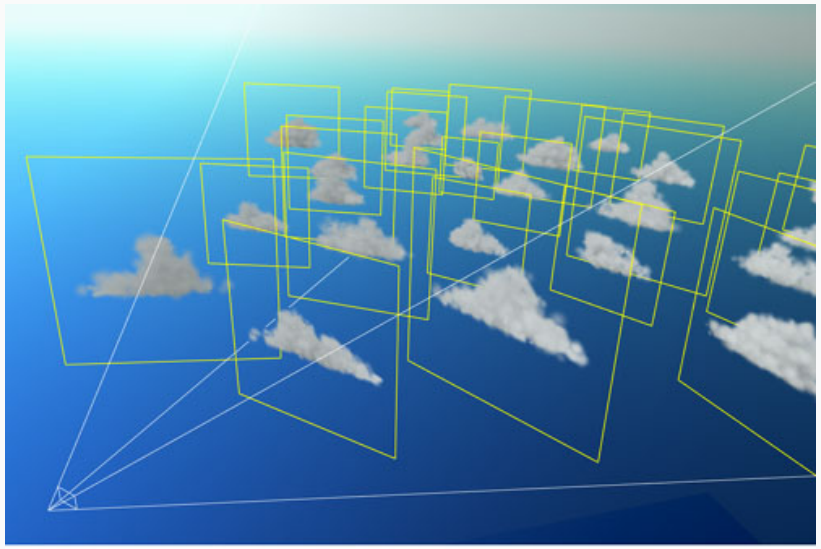
Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading'a
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью
- Визуализация нормалей
  - Вход: вершина с нормалью
  - Выход: линия из двух вершин – исходная вершина, исходная вершина + нормаль
- Shadow volumes – алгоритм рисования теней
- Billboards – плоские фигуры, всегда смотрящие в сторону камеры
  - Вход: одна вершина (точка)
  - Выход: набор треугольников
  - Системы частиц

Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading'a
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью
- Визуализация нормалей
  - Вход: вершина с нормалью
  - Выход: линия из двух вершин – исходная вершина, исходная вершина + нормаль
- Shadow volumes – алгоритм рисования теней
- Billboards – плоские фигуры, всегда смотрящие в сторону камеры
  - Вход: одна вершина (точка)
  - Выход: набор треугольников
  - Системы частиц
  - Облака

Billboards: облака





Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading'a
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью
- Визуализация нормалей
  - Вход: вершина с нормалью
  - Выход: линия из двух вершин – исходная вершина, исходная вершина + нормаль
- Shadow volumes – алгоритм рисования теней
- Billboards – плоские фигуры, всегда смотрящие в сторону камеры
  - Вход: одна вершина (точка)
  - Выход: набор треугольников
  - Системы частиц
  - Облака

Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading'a
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью
- Визуализация нормалей
  - Вход: вершина с нормалью
  - Выход: линия из двух вершин – исходная вершина, исходная вершина + нормаль
- Shadow volumes – алгоритм рисования теней
- Billboards – плоские фигуры, всегда смотрящие в сторону камеры
  - Вход: одна вершина (точка)
  - Выход: набор треугольников
  - Системы частиц
  - Облака
  - Далёкие деревья

Billboards: деревья



Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading'а
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью
- Визуализация нормалей
  - Вход: вершина с нормалью
  - Выход: линия из двух вершин – исходная вершина, исходная вершина + нормаль
- Shadow volumes – алгоритм рисования теней
- Billboards – плоские фигуры, всегда смотрящие в сторону камеры
  - Вход: одна вершина (точка)
  - Выход: набор треугольников
  - Системы частиц
  - Облака
  - Далёкие деревья

Геометрические шейдеры: примеры использования

- Расчёт нормалей для flat shading'a
  - Вход: треугольник (тройка вершин)
  - Выход: те же вершины с посчитанной нормалью
- Визуализация нормалей
  - Вход: вершина с нормалью
  - Выход: линия из двух вершин – исходная вершина, исходная вершина + нормаль
- Shadow volumes – алгоритм рисования теней
- Billboards – плоские фигуры, всегда смотрящие в сторону камеры
  - Вход: одна вершина (точка)
  - Выход: набор треугольников
  - Системы частиц
  - Облака
  - Далёкие деревья
  - Трава

Billboards: трава



Геометрические шейдеры: ссылки

- khronos.org/opengl/wiki/Geometry_Shader
- learnopengl.com/Advanced-OpenGL/Geometry-Shader
- open.gl/geometry
- lighthouse3d.com/tutorials/glsl-tutorial/geometry-shader
- GPU Gems, Chapter 7. Rendering Countless Blades of Waving Grass

Compute шейдеры

- Compute шейдеры – механизм для запуска произвольных параллельных вычислений на GPU, минующий весь графический конвейер

Compute шейдеры

- Compute шейдеры – механизм для запуска произвольных параллельных вычислений на GPU, минующий весь графический конвейер
- Для их запуска нужен не render pass, а *compute pass* (создаётся так же через command encoder)

Compute шейдеры

- Compute шейдеры – механизм для запуска произвольных параллельных вычислений на GPU, минующий весь графический конвейер
- Для их запуска нужен не render pass, а *compute pass* (создаётся так же через command encoder)
- Он не привязан к цветовым буферам или буферам глубины, для его создания не нужно почти никаких настроек

Compute шейдеры

- Compute шейдеры – механизм для запуска произвольных параллельных вычислений на GPU, минующий весь графический конвейер
- Для их запуска нужен не render pass, а *compute pass* (создаётся так же через command encoder)
- Он не привязан к цветовым буферам или буферам глубины, для его создания не нужно почти никаких настроек
- Так же как с render pass'ом, можно выставить bing группы и immediates

Compute pipeline

- Compute pass умеет использовать только *compute pipelines*

Compute pipeline

- Compute pass умеет использовать только *compute pipelines*
- Compute pipeline создаётся аналогично render pipeline'ам: нужен pipeline layout, shader module и entry point

Compute pipeline

- Compute pass умеет использовать только *compute pipelines*
- Compute pipeline создаётся аналогично render pipeline'ам: нужен pipeline layout, shader module и entry point
- Так как compute шейдеры не используют графический конвейер, связанных с ним настроек нет (vertex/fragment/primitive state и т.п.)

Compute pipeline

- Compute pass умеет использовать только *compute pipelines*
- Compute pipeline создаётся аналогично render pipeline'ам: нужен pipeline layout, shader module и entry point
- Так как compute шейдеры не используют графический конвейер, связанных с ним настроек нет (vertex/fragment/primitive state и т.п.)
- Команд `draw()` у compute pass'а нет, основной способ инициировать вычисление – `compute_pass.dispatch_workgroups()`

Compute шейдеры: модель исполнения

- Compute shader'ы запускаются сеткой $N_x \times N_y \times N_z$ ячеек, каждая ячейка соответствует одному потоку исполнения внутри GPU

Compute шейдеры: модель исполнения

- Compute shader'ы запускаются сеткой $N_x \times N_y \times N_z$ ячеек, каждая ячейка соответствует одному потоку исполнения внутри GPU
- GPU исполняют задачи SIMD-блоками (обычно 32 или 64 парралельных исполнения)

Compute шейдеры: модель исполнения

- Compute shader'ы запускаются сеткой $N_x \times N_y \times N_z$ ячеек, каждая ячейка соответствует одному потоку исполнения внутри GPU
- GPU исполняют задачи SIMD-блоками (обычно 32 или 64 парралельных исполнения)
- Compute шейдеры явно отражают это в своём интерфейсе: они запускаются как набор из нескольких *workgroup* одинакового размера $S_x \times S_y \times S_z$ (`workgroup_size`, задаётся в шейдере)

Compute шейдеры: модель исполнения

- Compute shader'ы запускаются сеткой $N_x \times N_y \times N_z$ ячеек, каждая ячейка соответствует одному потоку исполнения внутри GPU
- GPU исполняют задачи SIMD-блоками (обычно 32 или 64 парралельных исполнения)
- Compute шейдеры явно отражают это в своём интерфейсе: они запускаются как набор из нескольких *workgroup* одинакового размера $S_x \times S_y \times S_z$ (`workgroup_size`, задаётся в шейдере)
- Аргументы к `compute_pass.dispatch_workgroups()` – количество запущенных workgroup $C_x \times C_y \times C_z$ (`workgroup_count`)

Compute шейдеры: модель исполнения

- Compute shader'ы запускаются сеткой $N_x \times N_y \times N_z$ ячеек, каждая ячейка соответствует одному потоку исполнения внутри GPU
- GPU исполняют задачи SIMD-блоками (обычно 32 или 64 парралельных исполнения)
- Compute шейдеры явно отражают это в своём интерфейсе: они запускаются как набор из нескольких *workgroup* одинакового размера $S_x \times S_y \times S_z$ (`workgroup_size`, задаётся в шейдере)
- Аргументы к `compute_pass.dispatch_workgroups()` – количество запущенных workgroup $C_x \times C_y \times C_z$ (`workgroup_count`)
- В итоге, $N_x = S_x \cdot C_x$, и аналогично по другим осям

Compute шейдеры: модель исполнения

- Выбор размера workgroup – сложная задача:

Compute шейдеры: модель исполнения

- Выбор размера workgroup – сложная задача:
  - Один SIMD-блок GPU может выполнять только одну workgroup $\Longrightarrow$ в идеале размер workgroup должен быть кратен размеру SIMD-блока

Compute шейдеры: модель исполнения

- Выбор размера workgroup – сложная задача:
  - Один SIMD-блок GPU может выполнять только одну workgroup $\Longrightarrow$ в идеале размер workgroup должен быть кратен размеру SIMD-блока
  - Большой размер workgroup требует больше локальной памяти, но может позволить переиспользовать больше вычислений (через workgroup shared memory)

Compute шейдеры: модель исполнения

- Выбор размера workgroup – сложная задача:
  - Один SIMD-блок GPU может выполнять только одну workgroup $\Longrightarrow$ в идеале размер workgroup должен быть кратен размеру SIMD-блока
  - Большой размер workgroup требует больше локальной памяти, но может позволить переиспользовать больше вычислений (через workgroup shared memory)
  - Как слишком маленькими, так и слишком большими workgroup можно упереться в различные hardware лимиты

Compute шейдеры: модель исполнения

- Выбор размера workgroup – сложная задача:
  - Один SIMD-блок GPU может выполнять только одну workgroup $\Longrightarrow$ в идеале размер workgroup должен быть кратен размеру SIMD-блока
  - Большой размер workgroup требует больше локальной памяти, но может позволить переиспользовать больше вычислений (через workgroup shared memory)
  - Как слишком маленькими, так и слишком большими workgroup можно упереться в различные hardware лимиты
- Почти всегда SIMD-блоки имеют размер в 32 или 64 потока, поэтому стандартный выбор для workgroup size – 64 (произведение по всем осям)

Compute шейдеры: модель исполнения

- Выбор размера workgroup – сложная задача:
  - Один SIMD-блок GPU может выполнять только одну workgroup $\Longrightarrow$ в идеале размер workgroup должен быть кратен размеру SIMD-блока
  - Большой размер workgroup требует больше локальной памяти, но может позволить переиспользовать больше вычислений (через workgroup shared memory)
  - Как слишком маленькими, так и слишком большими workgroup можно упереться в различные hardware лимиты
- Почти всегда SIMD-блоки имеют размер в 32 или 64 потока, поэтому стандартный выбор для workgroup size – 64 (произведение по всем осям)
- Это удобно для обработки изображений (блоки 8×8) или 3D-данных (блоки $4 \times 4 \times 4$)

Compute шейдеры: модель исполнения

- Выбор размера workgroup – сложная задача:
  - Один SIMD-блок GPU может выполнять только одну workgroup $\Longrightarrow$ в идеале размер workgroup должен быть кратен размеру SIMD-блока
  - Большой размер workgroup требует больше локальной памяти, но может позволить переиспользовать больше вычислений (через workgroup shared memory)
  - Как слишком маленькими, так и слишком большими workgroup можно упереться в различные hardware лимиты
- Почти всегда SIMD-блоки имеют размер в 32 или 64 потока, поэтому стандартный выбор для workgroup size – 64 (произведение по всем осям)
- Это удобно для обработки изображений (блоки 8×8) или 3D-данных (блоки $4 \times 4 \times 4$)
- Если реальный объём работы (например, размер изображения) не кратен размеру workgroup, лишние вычисления обычно явно пропускаются в шейдере

Compute шейдеры: детали

- Entry point для compute pipeline (функция в шейдере) должна быть помечена атрибутами `@compute` и `@workgroup_size(x, y, z)` (`y` и `z` можно не указывать, если они равны 1)

Compute шейдеры: детали

- Entry point для compute pipeline (функция в шейдере) должна быть помечена атрибутами `@compute` и `@workgroup_size(x, y, z)` (`y` и `z` можно не указывать, если они равны 1)
- Встроенные переменные compute шейдера:
  - `@builtin(local_invocation_id)` – 3D индекс потока внутри workgroup (`0..workgroup_size-1`)
  - `@builtin(workgroup_id)` – 3D индекс workgroup внутри вызова dispatch (`0..workgroup_count-1`)
  - `@builtin(global_invocation_id)` – 3D индекс потока внутри вызова dispatch (вычисляется как `workgroup_id * workgroup_size + local_invocation_id`)
  - `@builtin(local_invocation_index)` – линеаризованный индекс потока внутри workgroup
  - `@builtin(global_invocation_index)` – линеаризованный индекс потока внутри dispatch

Compute шейдеры: детали

- Entry point для compute pipeline (функция в шейдере) должна быть помечена атрибутами `@compute` и `@workgroup_size(x, y, z)` (`y` и `z` можно не указывать, если они равны 1)
- Встроенные переменные compute шейдера:
  - `@builtin(local_invocation_id)` – 3D индекс потока внутри workgroup (`0..workgroup_size-1`)
  - `@builtin(workgroup_id)` – 3D индекс workgroup внутри вызова dispatch (`0..workgroup_count-1`)
  - `@builtin(global_invocation_id)` – 3D индекс потока внутри вызова dispatch (вычисляется как `workgroup_id * workgroup_size + local_invocation_id`)
  - `@builtin(local_invocation_index)` – линеаризованный индекс потока внутри workgroup
  - `@builtin(global_invocation_index)` – линеаризованный индекс потока внутри dispatch
- Встроенных входных или выходных данных нет, все операции делаются через данные, переданные в bind groups

Compute шейдеры: пример

```
@group(0) @binding(0) var<storage, read> a: array<f32>;
@group(0) @binding(1) var<storage, read> b: array<f32>;
@group(0) @binding(2) var<storage, read_write> c: array<f32>;

@compute @workgroup_size(64)
fn main(@builtin(global_invocation_id) id: vec3u) {
    let index = id.x;
    if (index < arrayLength(&result)) {
        c[index] = a[index] + b[index];
    }
}
```

Compute шейдеры: пример

```
@group(0) @binding(0) var image:
  texture_storage_2d<rgba8unorm, read_write>;

@compute @workgroup_size(8, 8, 1)
fn main(@builtin(global_invocation_id) id: vec3u) {
    if (any(id.xy >= textureDimensions(imgOutput))) {
      return;
    }

    let colorIn = textureLoad(image, id.xy);

    let colorOut = vec4f(vec3f(1.0) - colorIn.rgb, colorIn.a);

    textureStore(image, id.xy, colorOut);
}
```

Storage buffers

- Буферы, созданные с флагом `usage = STORAGE`

Storage buffers

- Буферы, созданные с флагом `usage = STORAGE`
- Их можно добавить в bind группу с режимом `storage` или `read_only_storage`

Storage buffers

- Буферы, созданные с флагом `usage = STORAGE`
- Их можно добавить в bind группу с режимом `storage` или `read_only_storage`
- В шейдере должны быть объявлены как `var<storage, read>` или `var<storage, read_write>`, соответственно

Storage buffers

- Буферы, созданные с флагом `usage = STORAGE`
- Их можно добавить в bind группу с режимом `storage` или `read_only_storage`
- В шейдере должны быть объявлены как `var<storage, read>` или `var<storage, read_write>`, соответственно
- Тип данных – любой, в том числе может быть массив неизвестного размера: `array<vec4f>` (размер массива можно узнать через функцию `arrayLength`)

Storage buffers

- Буферы, созданные с флагом `usage = STORAGE`
- Их можно добавить в bind группу с режимом `storage` или `read_only_storage`
- В шейдере должны быть объявлены как `var<storage, read>` или `var<storage, read_write>`, соответственно
- Тип данных – любой, в том числе может быть массив неизвестного размера: `array<vec4f>` (размер массива можно узнать через функцию `arrayLength`)
- Основной способ передать произвольные данные на GPU, размер ограничен размером видеопамяти

Storage buffers

- Буферы, созданные с флагом `usage = STORAGE`
- Их можно добавить в bind группу с режимом `storage` или `read_only_storage`
- В шейдере должны быть объявлены как `var<storage, read>` или `var<storage, read_write>`, соответственно
- Тип данных – любой, в том числе может быть массив неизвестного размера: `array<vec4f>` (размер массива можно узнать через функцию `arrayLength`)
- Основной способ передать произвольные данные на GPU, размер ограничен размером видеопамяти
- Не read-only буферы можно использовать только во фрагментных и compute шейдерах (с фичей `VERTEX_WRITABLE_STORAGE` можно и в вершинных – не поддерживается в браузерной версии WebGPU)

Storage textures

- Текстуры, созданные с флагом `usage = STORAGE_BINDING`

Storage textures

- Текстуры, созданные с флагом `usage = STORAGE_BINDING`
- Их тоже можно добавить в bind группу с явным указанием формата

Storage textures

- Текстуры, созданные с флагом `usage = STORAGE_BINDING`
- Их тоже можно добавить в bind группу с явным указанием формата
- В шейдере тоже объявляются с явным указанием формата, например `texture_storage_2d<rgba8unorm, read_write>`

Storage textures

- Текстуры, созданные с флагом `usage = STORAGE_BINDING`
- Их тоже можно добавить в bind группу с явным указанием формата
- В шейдере тоже объявляются с явным указанием формата, например `texture_storage_2d<rgba8unorm, read_write>`
- Нельзя использовать вместе с семплерами, только читать и писать конкретные пиксели через `textureLoad` и `textureStore`

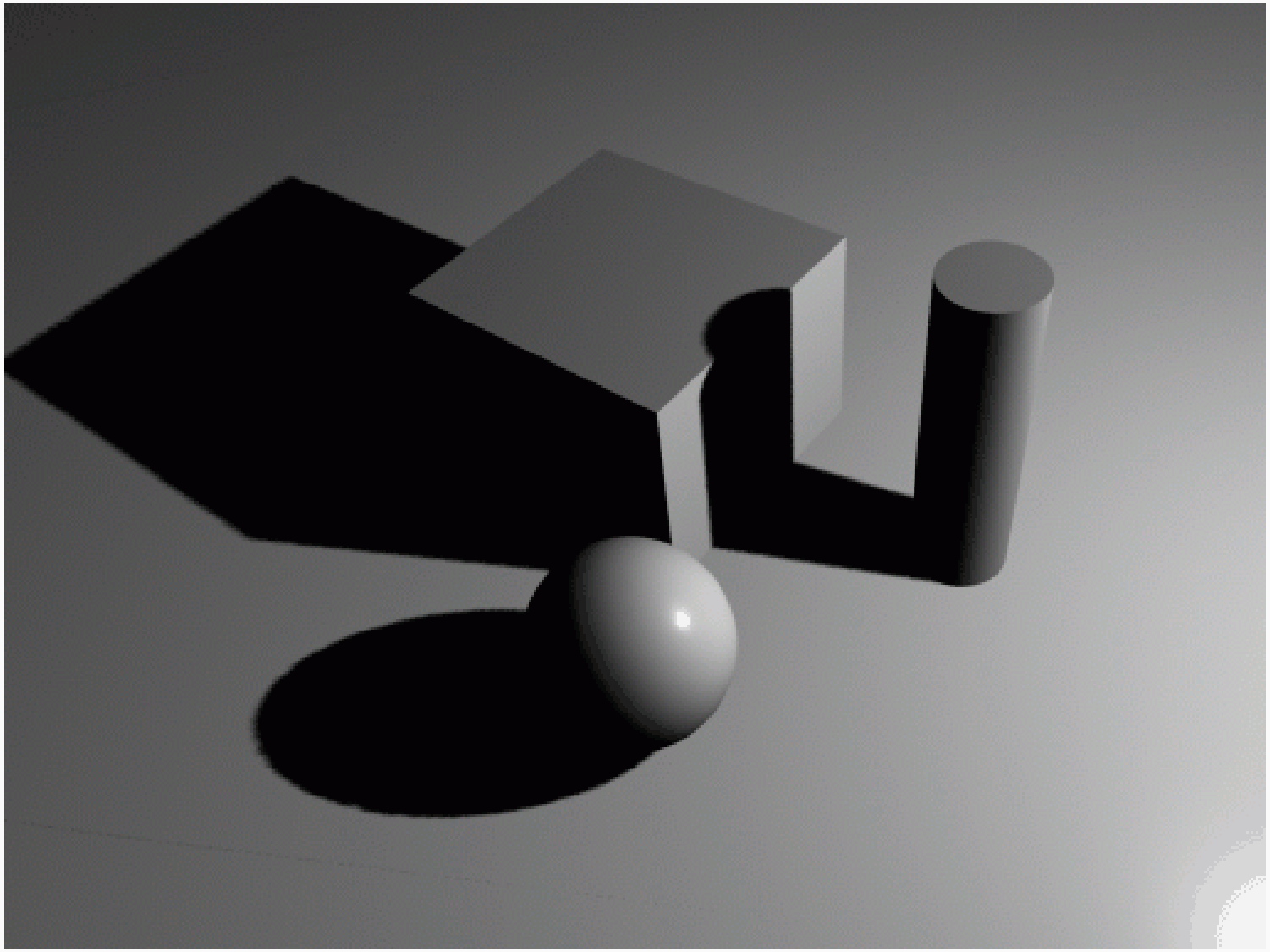
Storage textures

- Текстуры, созданные с флагом `usage = STORAGE_BINDING`
- Их тоже можно добавить в bind группу с явным указанием формата
- В шейдере тоже объявляются с явным указанием формата, например `texture_storage_2d<rgba8unorm, read_write>`
- Нельзя использовать вместе с семплерами, только читать и писать конкретные пиксели через `textureLoad` и `textureStore`
- Поддерживают ограниченный набор форматов

Тени: теория

- Точка сцены, в которую не попадает (заблокирован чем-то) прямой свет из конкретного источника света
- Свойство точки по отношению к конкретному источнику света

Тени



Тени: теория

- Если источник света точечный (или бесконечно удалённый), тень – бинарное свойство: луч из точки сцены в источник света или пересекает что-то (точка в тени), или нет (точка не в тени) $\Longrightarrow$ жёсткие тени (*hard shadows*)

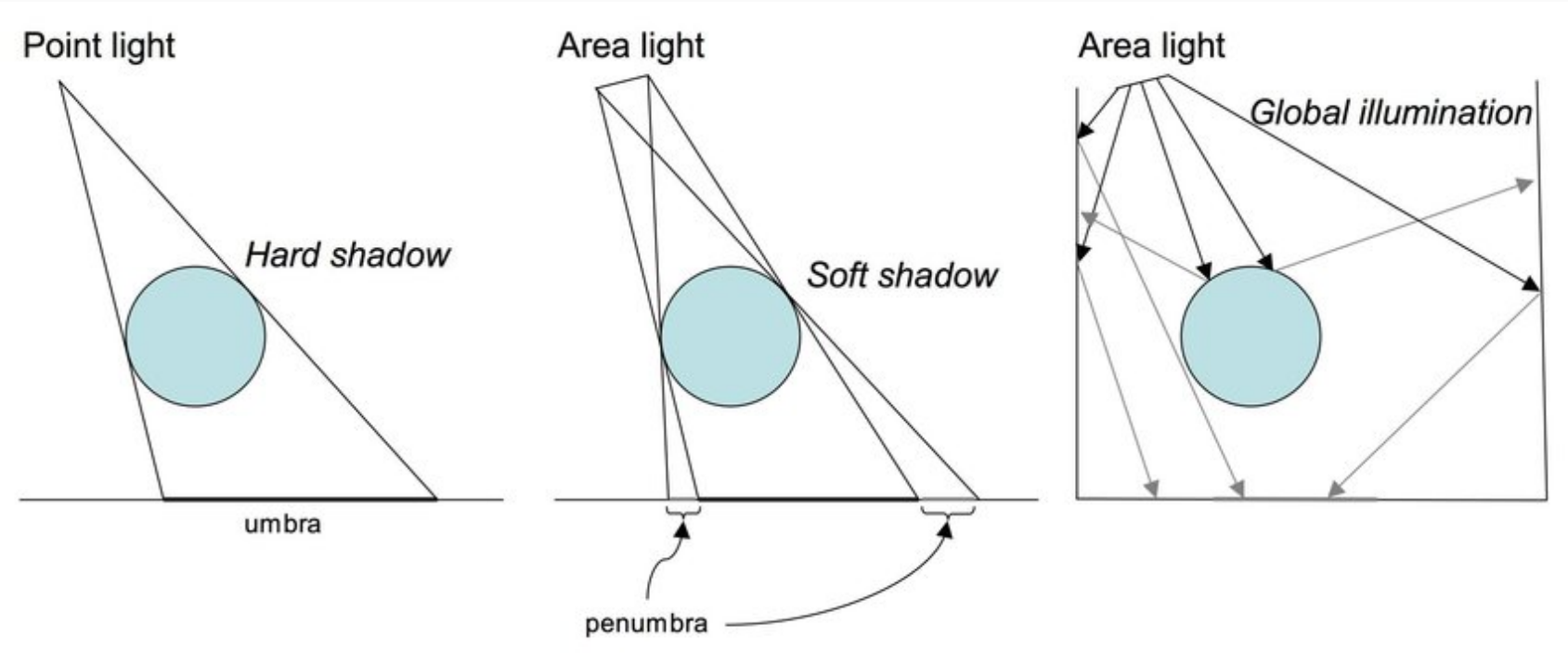
Тени: теория

- Если источник света точечный (или бесконечно удалённый), тень – бинарное свойство: луч из точки сцены в источник света или пересекает что-то (точка в тени), или нет (точка не в тени) $\Longrightarrow$ жёсткие тени (*hard shadows*)
- Если источник света объёмный, точка сцены может находиться в тени относительно части источника света, и не находиться в тени относительно другой его части $\Longrightarrow$ мягкие тени (*soft shadows*)

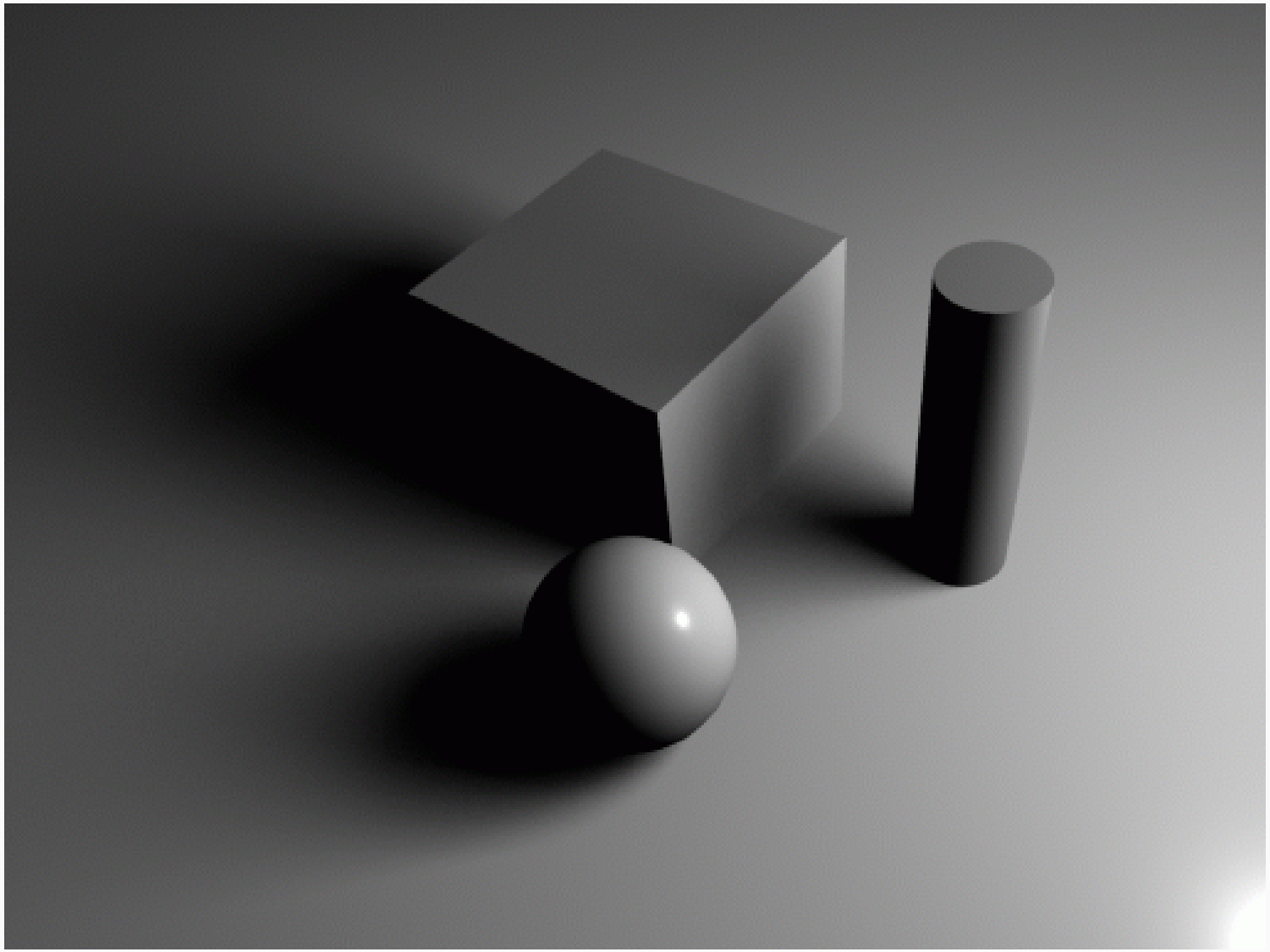
Тени: теория

- Если источник света точечный (или бесконечно удалённый), тень – бинарное свойство: луч из точки сцены в источник света или пересекает что-то (точка в тени), или нет (точка не в тени) $\Longrightarrow$ жёсткие тени (*hard shadows*)
- Если источник света объёмный, точка сцены может находиться в тени относительно части источника света, и не находиться в тени относительно другой его части $\Longrightarrow$ мягкие тени (*soft shadows*)
  - Точки, полностью находящиеся в тени – *umbra*
  - Точки, частично находящиеся в тени – *penumbra* (*полутень*)

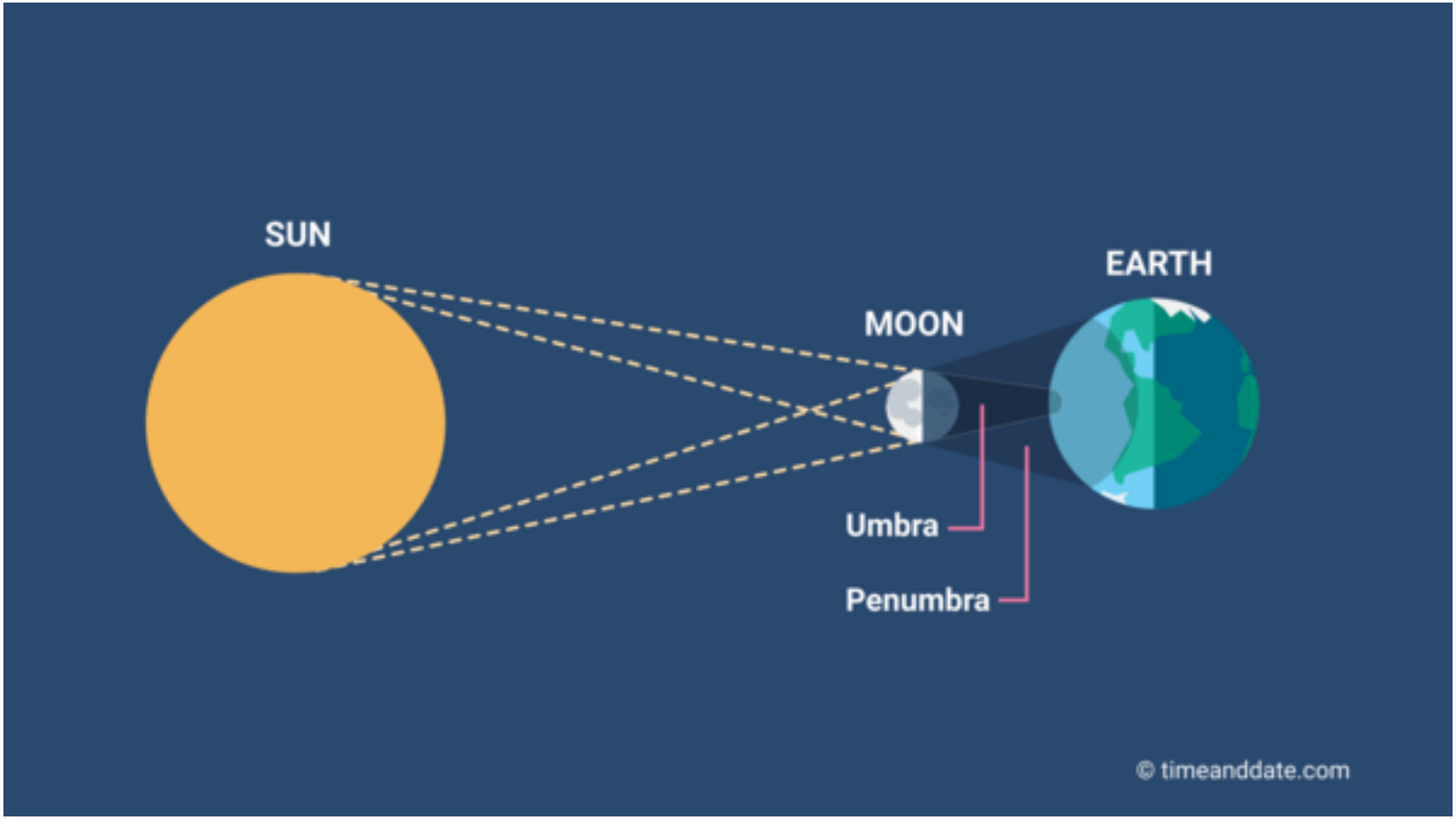
Тени: мягкие vs жёсткие



Мягкие тени



Солнечное затмение



Тени: теория

- Реальные источники света – объёмные

Тени: теория

- Реальные источники света – объёмные
- Размер и форма penumbra зависит от размеров и формы источника света, а также от расстояния до него (чем дальше, тем меньше penumbra)

Тени: теория

- Реальные источники света – объёмные
- Размер и форма penumbra зависит от размеров и формы источника света, а также от расстояния до него (чем дальше, тем меньше penumbra)
  - В пределе расстояния $\to \infty$ мягкая тень вырождается в жёсткую

Тени: теория

- Реальные источники света – объёмные
- Размер и форма penumbra зависит от размеров и формы источника света, а также от расстояния до него (чем дальше, тем меньше penumbra)
  - В пределе расстояния $\to \infty$ мягкая тень вырождается в жёсткую
- В real-time графике получить правильные тени (как жёсткие, так и мягкие) довольно сложно

Тени: raytracing

- При простом raytracing'е (*Whitted-style*, без полного решения rendering equation) для получения жёсткой тени можно послать дополнительный луч в направлении света (shadow ray): если он что-то пересёк, точка находится в тени

Тени: raytracing

- При простом raytracing'е (*Whitted-style*, без полного решения rendering equation) для получения жёсткой тени можно послать дополнительный луч в направлении света (shadow ray): если он что-то пересёк, точка находится в тени
- При raytracing'е с монте-карло интегрированием для global illumination тени (как жёсткие, так и мягкие) получаются автоматически

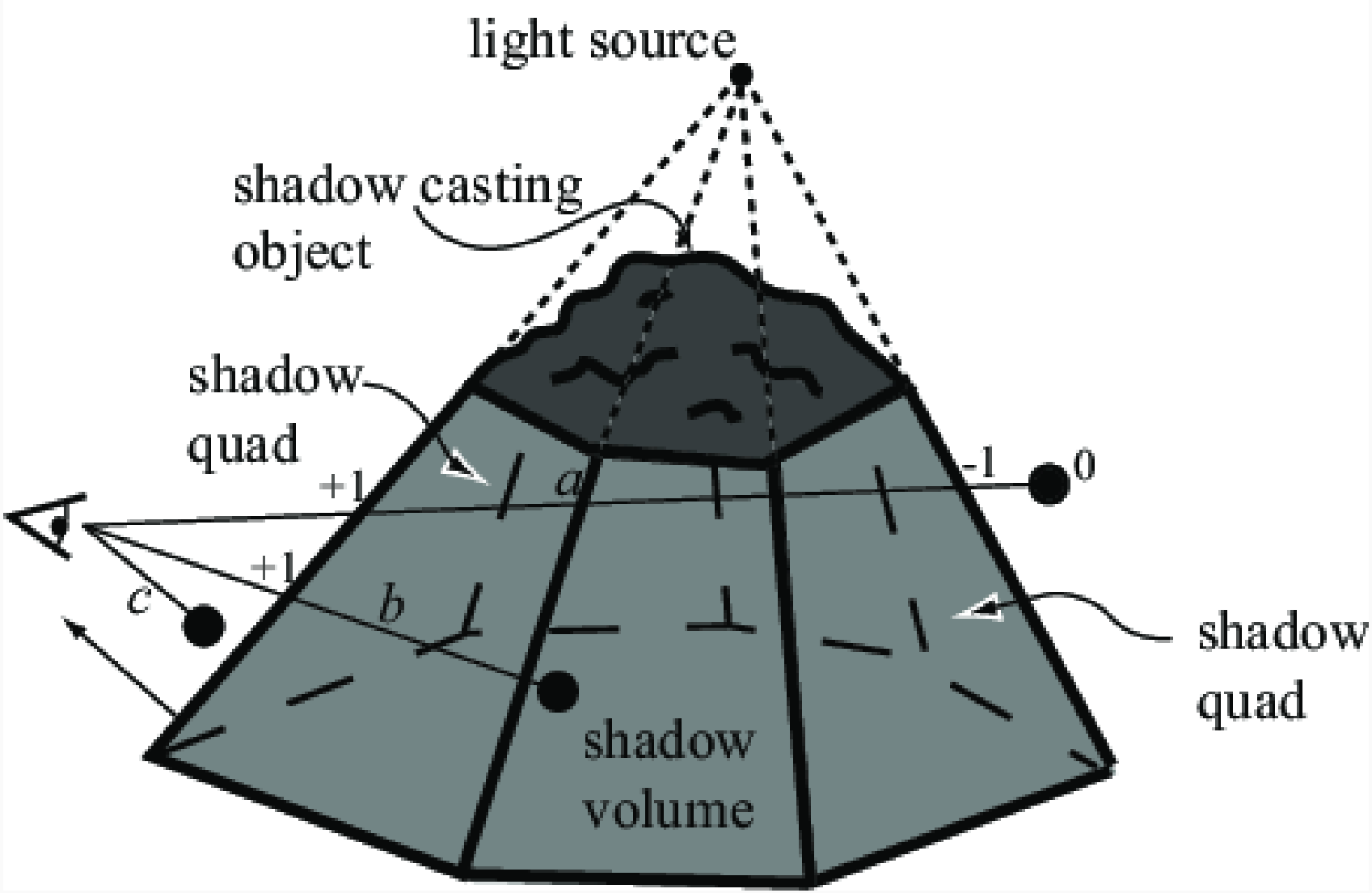
Тени: real-time алгоритмы

- Shadow volumes
- Shadow mapping
- Raytraced shadows

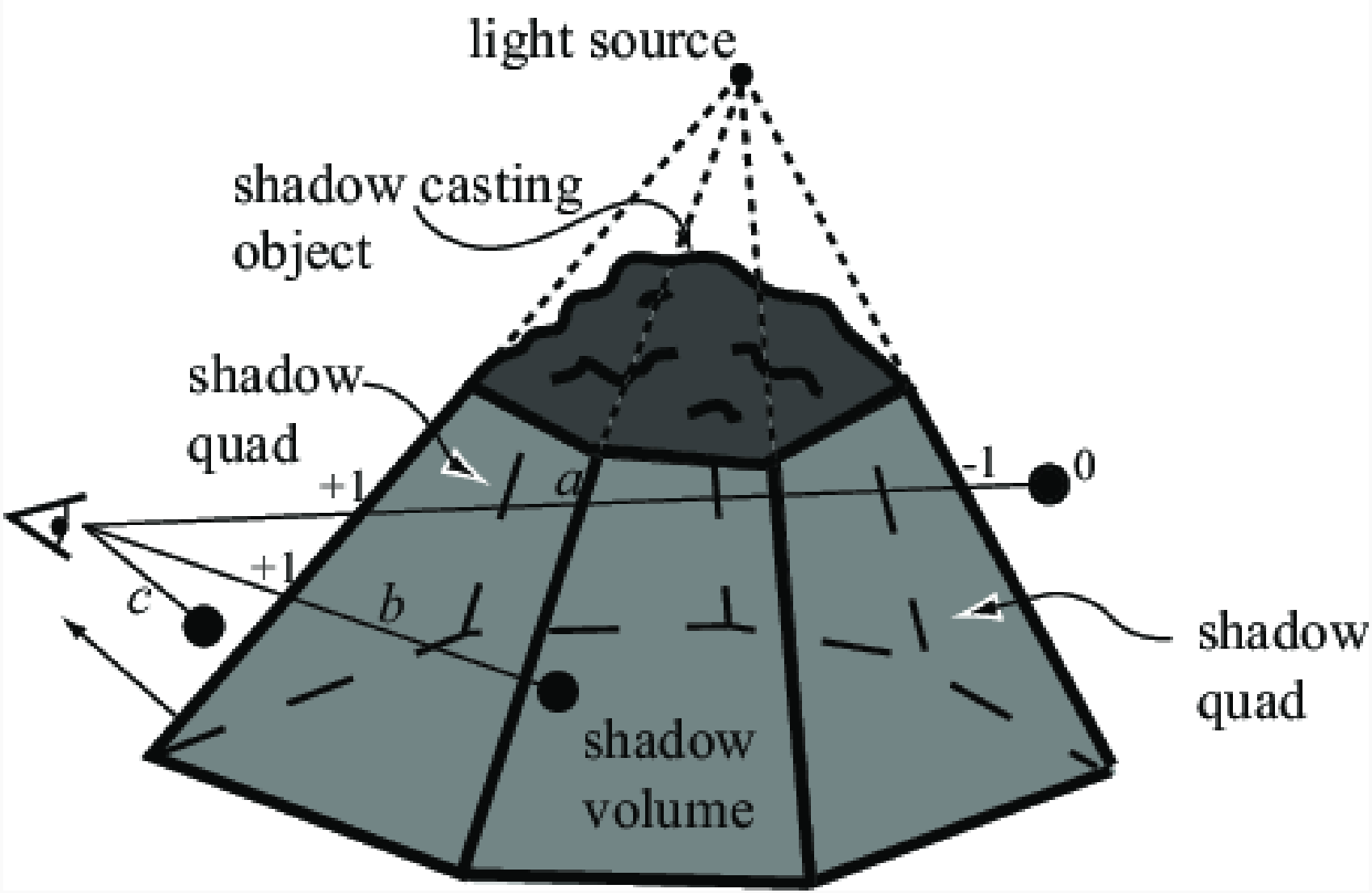
Shadow volumes (stencil shadows)

- Тень от конкретного объекта – некая (полубесконечная) трёхмерная область пространства (shadow volume)

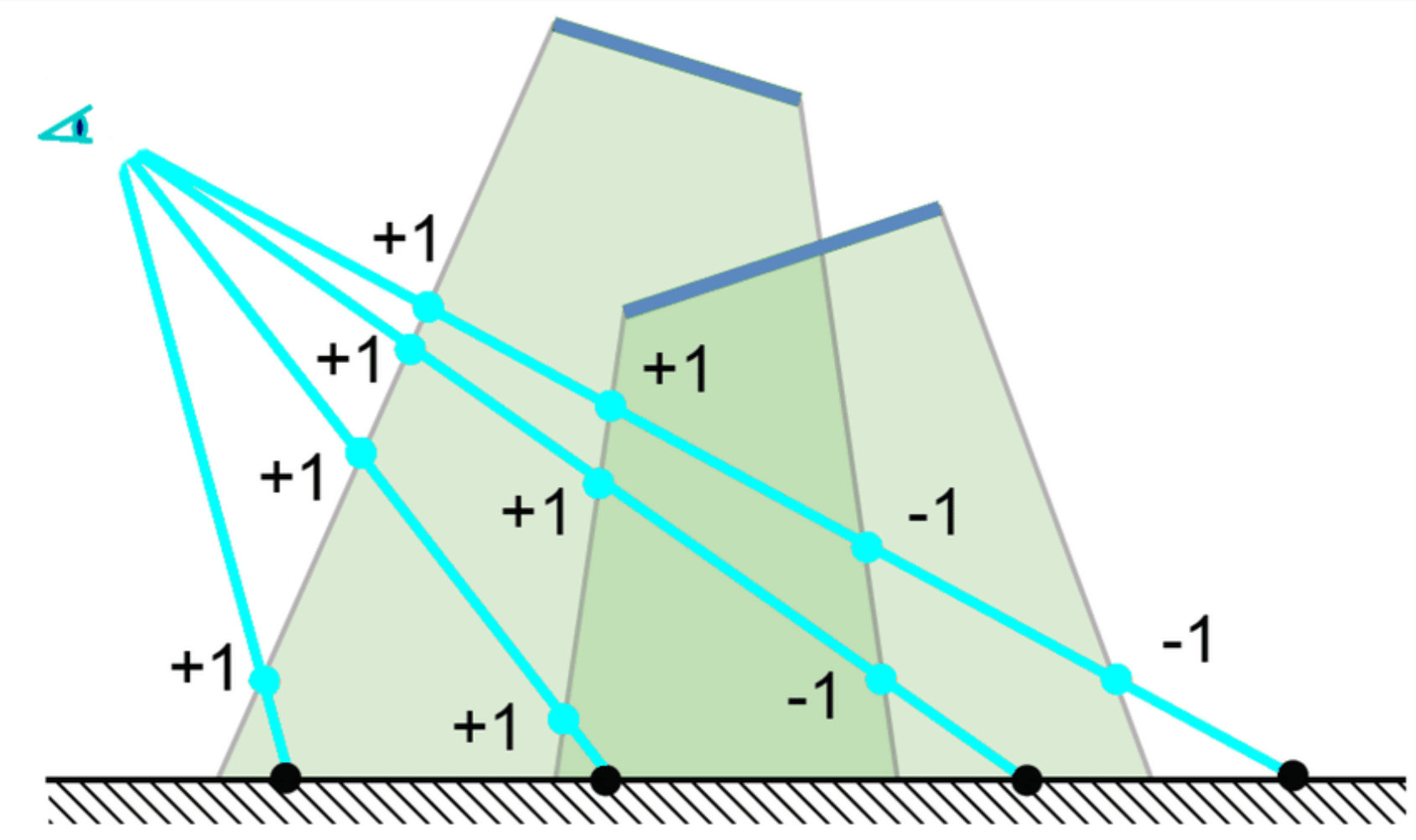
Shadow volumes (stencil shadows)



Shadow volumes (stencil shadows)



Shadow volumes (stencil shadows)



Shadow volumes: алгоритм

- Выключаем рисование в цветовой буфер и буфер глубины (как настройки render pipeline, или как отдельный render pass), но **не выключаем** тест глубины

Shadow volumes: алгоритм

- Выключаем рисование в цветовой буфер и буфер глубины (как настройки render pipeline, или как отдельный render pass), но **не выключаем** тест глубины
- Настраиваем запись в stencil буфер: `+1` для front-facing треугольников, `-1` для back-facing

Shadow volumes: алгоритм

- Выключаем рисование в цветовой буфер и буфер глубины (как настройки render pipeline, или как отдельный render pass), но **не выключаем** тест глубины
- Настраиваем запись в stencil буфер: `+1` для front-facing треугольников, `-1` для back-facing
- Вычисляем и рисуем shadow volume для каждого объекта

Shadow volumes: алгоритм

- Выключаем рисование в цветовой буфер и буфер глубины (как настройки render pipeline, или как отдельный render pass), но **не выключаем** тест глубины
- Настраиваем запись в stencil буфер: `+1` для front-facing треугольников, `-1` для back-facing
- Вычисляем и рисуем shadow volume для каждого объекта
- Обратно включаем рисование цвета и глубины

Shadow volumes: алгоритм

- Выключаем рисование в цветовой буфер и буфер глубины (как настройки render pipeline, или как отдельный render pass), но **не выключаем** тест глубины
- Настраиваем запись в stencil буфер: `+1` для front-facing треугольников, `-1` для back-facing
- Вычисляем и рисуем shadow volume для каждого объекта
- Обратно включаем рисование цвета и глубины
- Настраиваем stencil-тест так, чтобы рисовать только пиксели, в которых `stencil == 0`, и рисуем сцену шейдером, вычисляющим освещение

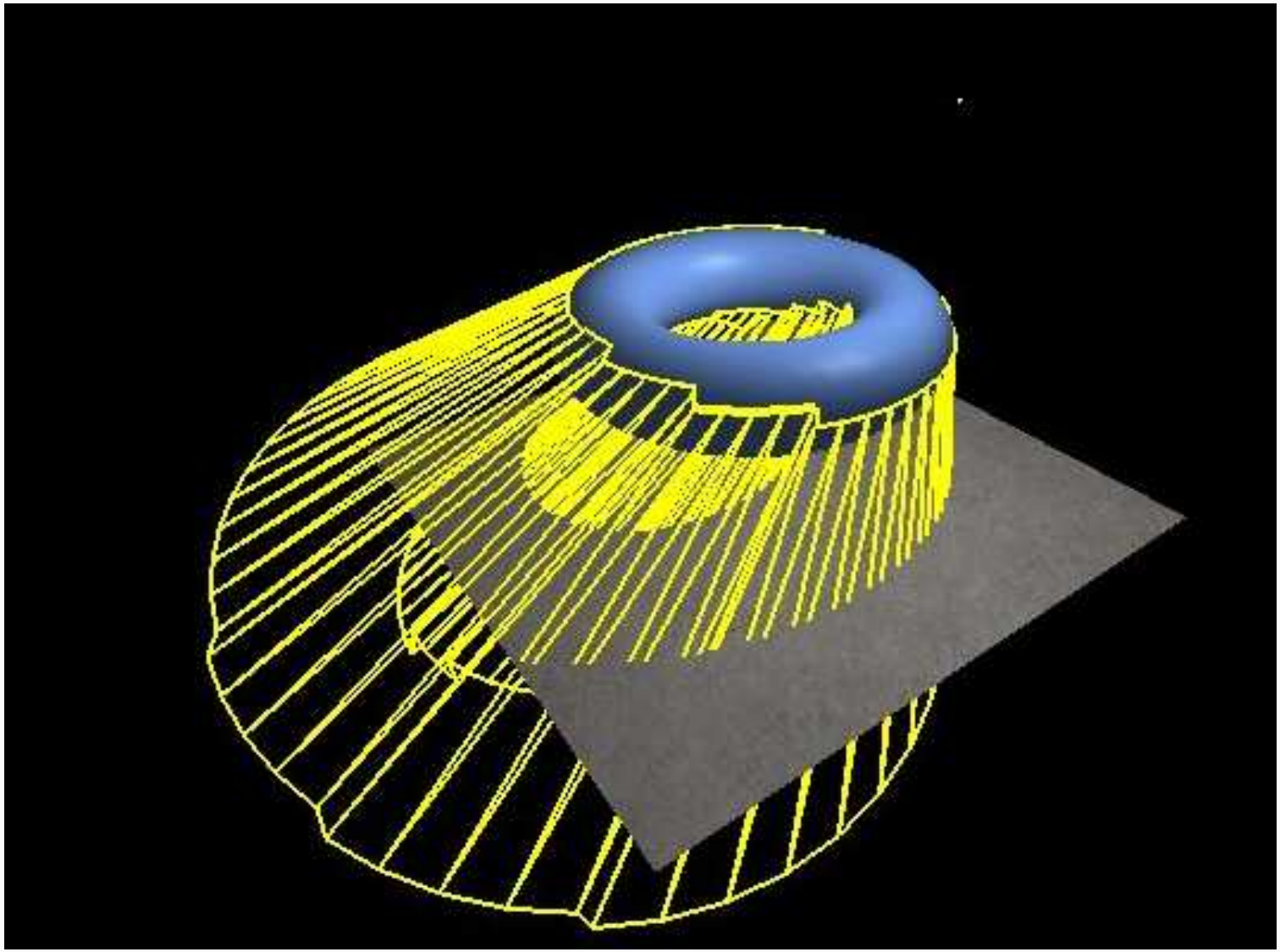
Shadow volumes: алгоритм

- Выключаем рисование в цветовой буфер и буфер глубины (как настройки render pipeline, или как отдельный render pass), но **не выключаем** тест глубины
- Настраиваем запись в stencil буфер: `+1` для front-facing треугольников, `-1` для back-facing
- Вычисляем и рисуем shadow volume для каждого объекта
- Обратно включаем рисование цвета и глубины
- Настраиваем stencil-тест так, чтобы рисовать только пиксели, в которых `stencil == 0`, и рисуем сцену шейдером, вычисляющим освещение
- Настраиваем stencil-тест так, чтобы рисовать только пиксели, в которых `stencil != 0`, и рисуем сцену шейдером, игнорирующим освещение

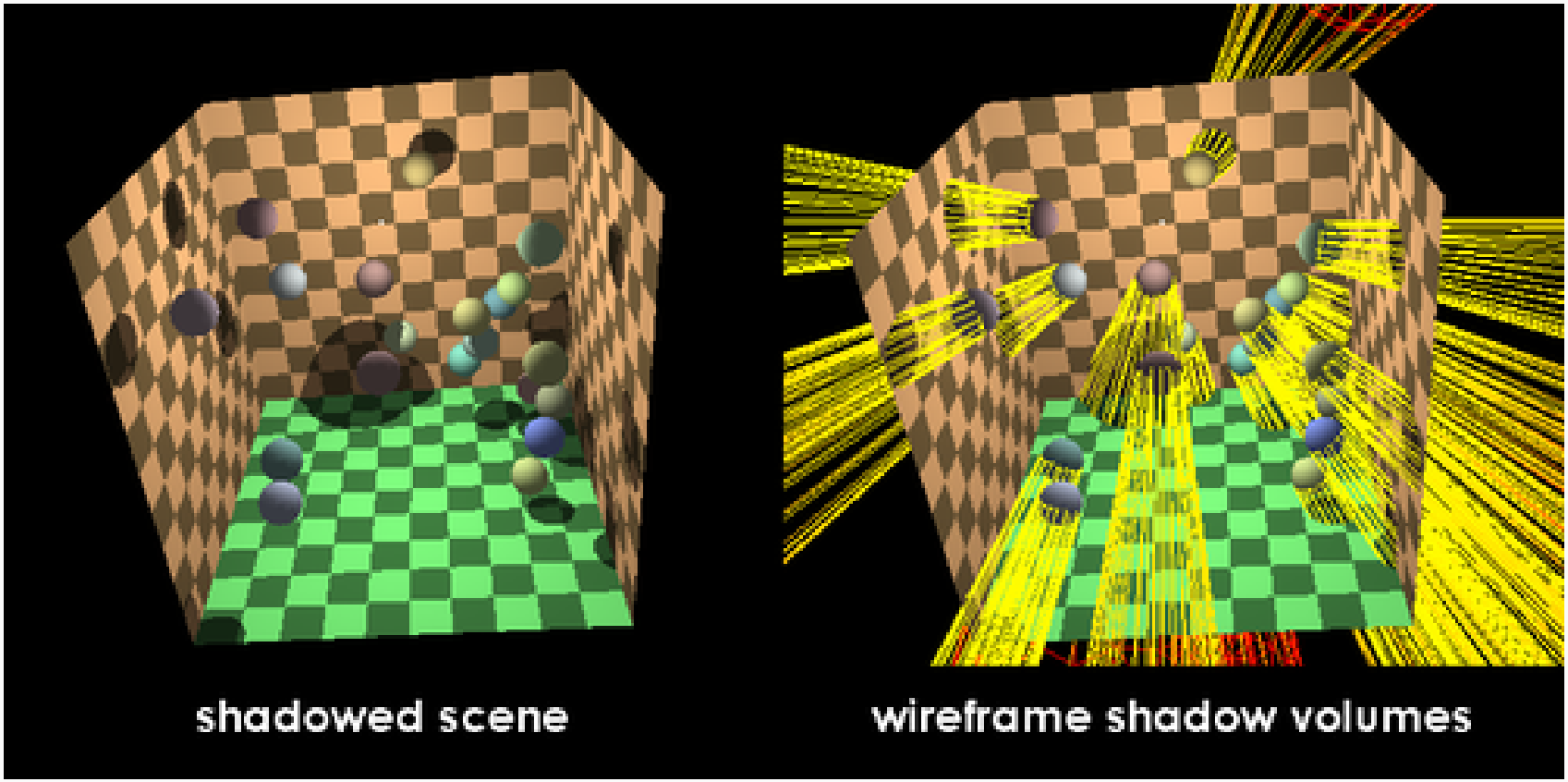
Shadow volumes: алгоритм

- Выключаем рисование в цветовой буфер и буфер глубины (как настройки render pipeline, или как отдельный render pass), но **не выключаем** тест глубины
- Настраиваем запись в stencil буфер: `+1` для front-facing треугольников, `-1` для back-facing
- Вычисляем и рисуем shadow volume для каждого объекта
- Обратно включаем рисование цвета и глубины
- Настраиваем stencil-тест так, чтобы рисовать только пиксели, в которых `stencil == 0`, и рисуем сцену шейдером, вычисляющим освещение
- Настраиваем stencil-тест так, чтобы рисовать только пиксели, в которых `stencil != 0`, и рисуем сцену шейдером, игнорирующим освещение
- Повторяем для каждого источника света

Shadow volume



Shadow volume



Shadow volumes (Doom 3)



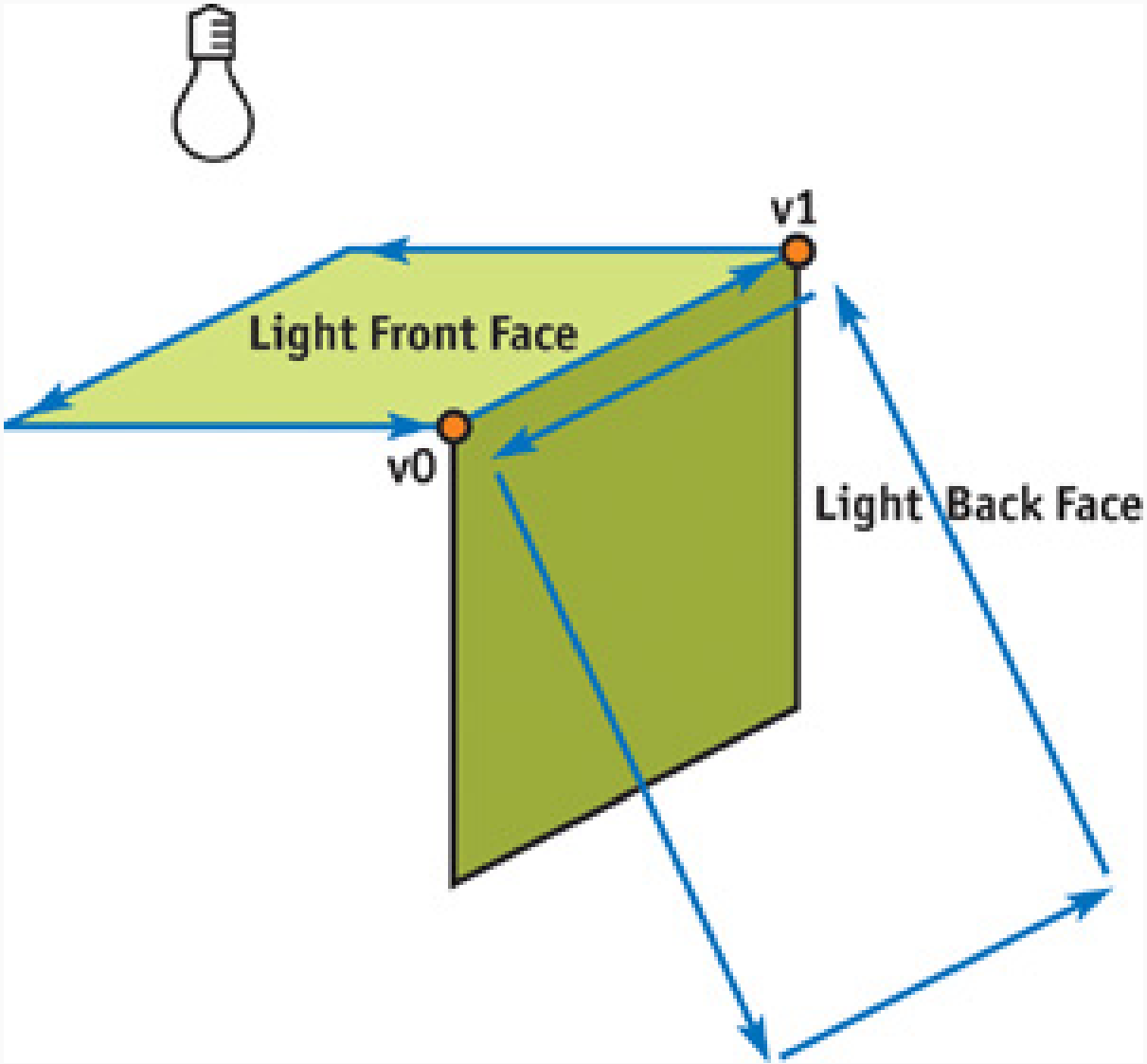
Shadow volumes: как вычислять shadow volume?

- Шаг 1: определить shadow edges – рёбра модели, у которых один соседний треугольник обращён к источнику света $\left(\vec{n} \cdot \vec{r} > 0\right)$, а второй – против источника света $\left(\vec{n} \cdot \vec{r} \leq 0\right)$

Shadow volumes: как вычислять shadow volume?

- Шаг 1: определить shadow edges – рёбра модели, у которых один соседний треугольник обращён к источнику света $\left(\vec{n} \cdot \vec{r} > 0\right)$, а второй – против источника света $\left(\vec{n} \cdot \vec{r} \leq 0\right)$
- Шаг 2: для каждого shadow edge построить четырёхугольную грань shadow volume – к точкам ребра p_1, p_2 добавлются точки $p_1 + D \cdot \frac{p_1 - o}{|p_1 - o|}$ и аналогично для p_2
  - o – координаты источника света
  - D – расстояние, до которого отбрасывается тень (в идеале мы хотим $D \to \infty$, для этого можно воспользоваться однородными координатами)

Shadow volumes (Doom 3)



Shadow volumes: как вычислять shadow volume?

- Два варианта: на CPU или на GPU

Shadow volumes: как вычислять shadow volume?

- Два варианта: на CPU или на GPU
- На CPU: очень дорого (положение модели и источника света могут меняться каждый кадр)

Shadow volumes: как вычислять shadow volume?

- Два варианта: на CPU или на GPU
- На CPU: очень дорого (положение модели и источника света могут меняться каждый кадр)
- На GPU – много вариантов (вершинный/геометрический/compute шейдер), главная проблема – передать в шейдер две вершины одного ребра + две вершины соседних треугольников

Shadow volumes: как вычислять shadow volume?

- Два варианта: на CPU или на GPU
- На CPU: очень дорого (положение модели и источника света могут меняться каждый кадр)
- На GPU – много вариантов (вершинный/геометрический/compute шейдер), главная проблема – передать в шейдер две вершины одного ребра + две вершины соседних треугольников
- В любом случае нужны входные данные, специально адаптированные для генерации shadow volume

Shadow volumes: как вычислять shadow volume?

- Два варианта: на CPU или на GPU
- На CPU: очень дорого (положение модели и источника света могут меняться каждый кадр)
- На GPU – много вариантов (вершинный/геометрический/compute шейдер), главная проблема – передать в шейдер две вершины одного ребра + две вершины соседних треугольников
- В любом случае нужны входные данные, специально адаптированные для генерации shadow volume
- Шейдер проверяет, является ли ребро shadow edge, и генерирует грань shadow volume

Shadow volumes

- \+ Pixel-perfect жёсткие тени
- — Алиасинг
- — Сложно получить мягкие тени
- — Нужно на лету генерировать геометрию
- — Требует правильной входной геометрии (без дырок и дублирования)
- — Не работает, если камера находится внутри тени (исправляется с помощью т.н. Carmack's Reverse)
- — Ресурсозатратен: даже для маленького объекта его shadow volume может занимать значительную часть сцены $\Longrightarrow$ слишком много растеризации, чтения/записи памяти (stencil buffer)

Shadow volumes с плохой геометрией



Shadow volumes

- Thief 3
- Doom 3
- Quake 4
- Prey
- Far Cry
- F.E.A.R 1, 2, 3
- S.T.A.L.K.E.R.
- Timeshift
- и др.

Shadow volumes: ссылки

- en.wikipedia.org/wiki/Shadow_volume
- ogldev.org/www/tutorial40/tutorial40.html
- GPU Gems, Chapter 9. Efficient Shadow Volume Rendering

Shadow mapping: идея

- Самое популярное семейство алгоритмов рисования теней

Shadow mapping: идея

- Самое популярное семейство алгоритмов рисования теней
- Идея: представим, что источник света – это камера (наблюдатель)
- Тень – это то, что *не видит* источник света

Shadow mapping: идея

- Самое популярное семейство алгоритмов рисования теней
- Идея: представим, что источник света – это камера (наблюдатель)
- Тень – это то, что *не видит* источник света
- Как понять, что видит источник света?

Shadow mapping: идея

- Самое популярное семейство алгоритмов рисования теней
- Идея: представим, что источник света – это камера (наблюдатель)
- Тень – это то, что *не видит* источник света
- Как понять, что видит источник света? Нарисовать!

Shadow mapping: алгоритм

- Рисуем сцену в текстуру (*shadow map*, карта теней) с точки зрения источника света (цветовой буфер не нужен, достаточно буфера глубины)

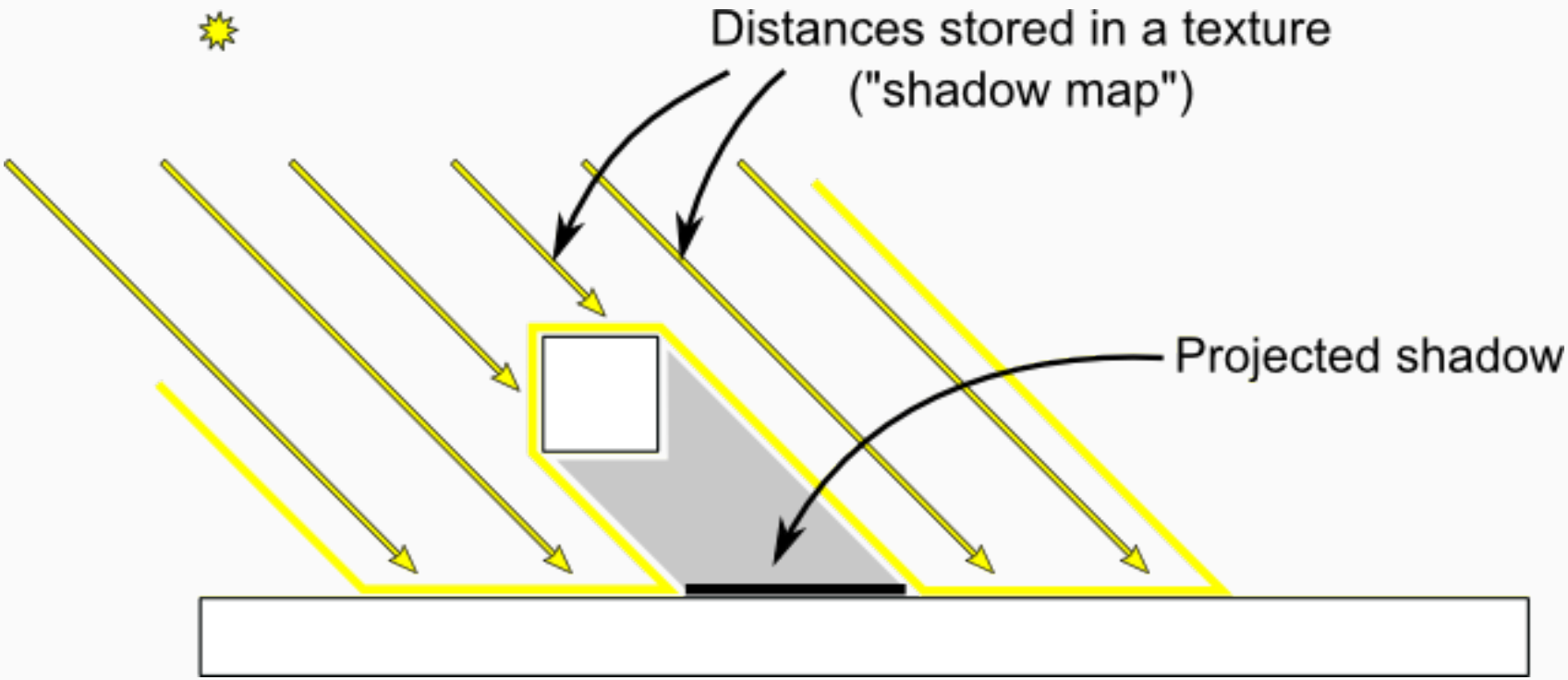
Shadow mapping: алгоритм

- Рисуем сцену в текстуру (*shadow map*, карта теней) с точки зрения источника света (цветовой буфер не нужен, достаточно буфера глубины)
- Рисуем сцену на экран, фрагментный шейдер читает *буфер глубины* (shadow map), нарисованный с точки зрения источника света – там содержатся расстояния от источника до ближайшей (к источнику) поверхности
- Если расстояние от источника до рисуемого пикселя больше, чем значение из буфера глубины, – текущий пиксель находится в тени

Shadow mapping: алгоритм

- Рисуем сцену в текстуру (*shadow map*, карта теней) с точки зрения источника света (цветовой буфер не нужен, достаточно буфера глубины)
- Рисуем сцену на экран, фрагментный шейдер читает *буфер глубины* (shadow map), нарисованный с точки зрения источника света – там содержатся расстояния от источника до ближайшей (к источнику) поверхности
- Если расстояние от источника до рисуемого пикселя больше, чем значение из буфера глубины, – текущий пиксель находится в тени
- N.B. Линейная фильтрация для буфера глубины не имеет смысла – будут артефакты на границах объектов

Shadow mapping: алгоритм



Shadow mapping: фрагментный шейдер

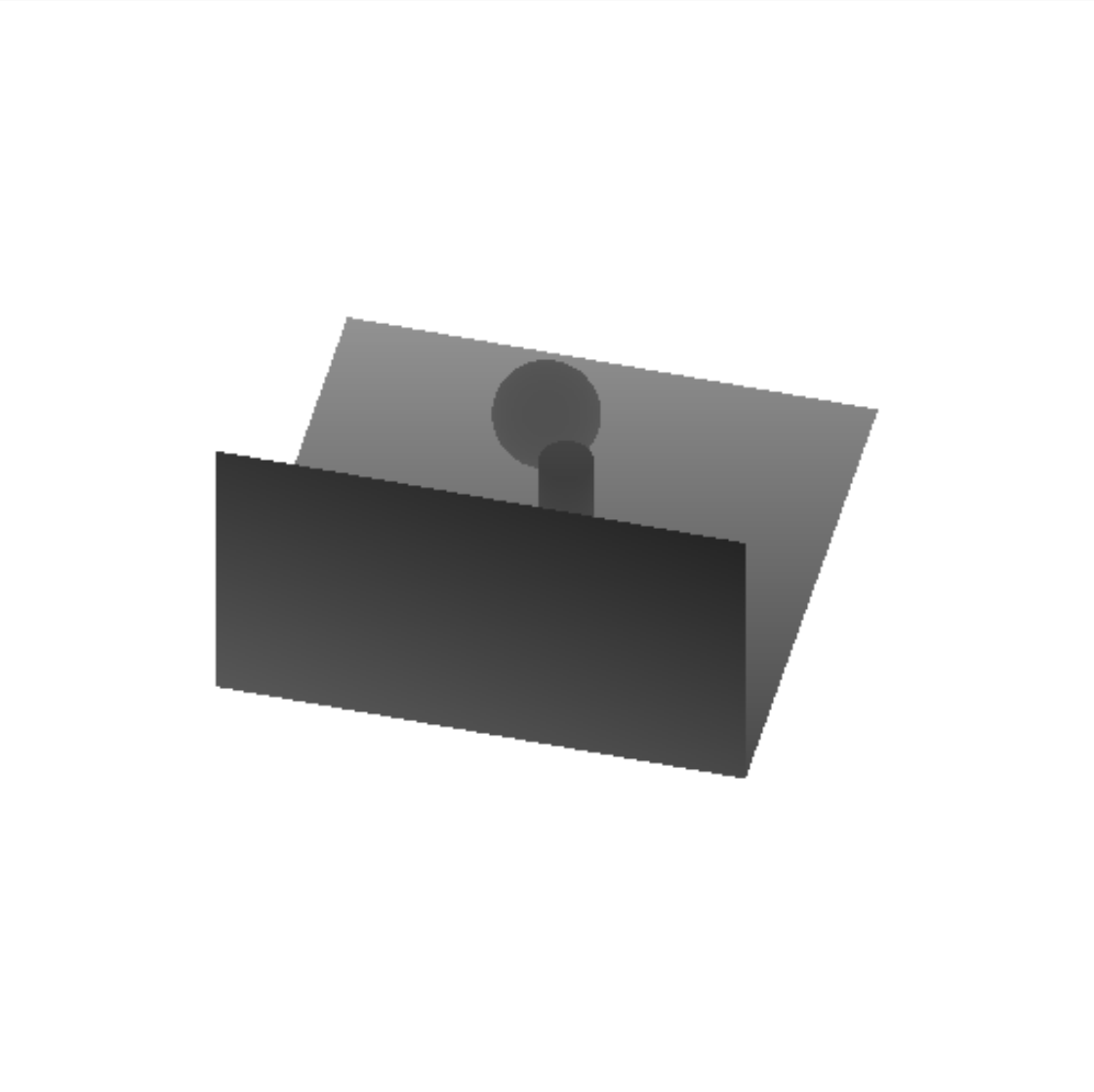
```
@group(0) @binding(0) var<uniform> shadow_projection: mat4x4f;
@group(0) @binding(1) var shadow_map: texture_depth_2d;
@group(0) @binding(2) var shadow_map_sampler: sampler;

@fragment
fn fragmentMain(in: VertexOut) -> @location(0) vec4f {
  var shadow_coord = shadow_projection * in.world_position;
  shadow_coord /= shadow_coord.w; // perspective divide

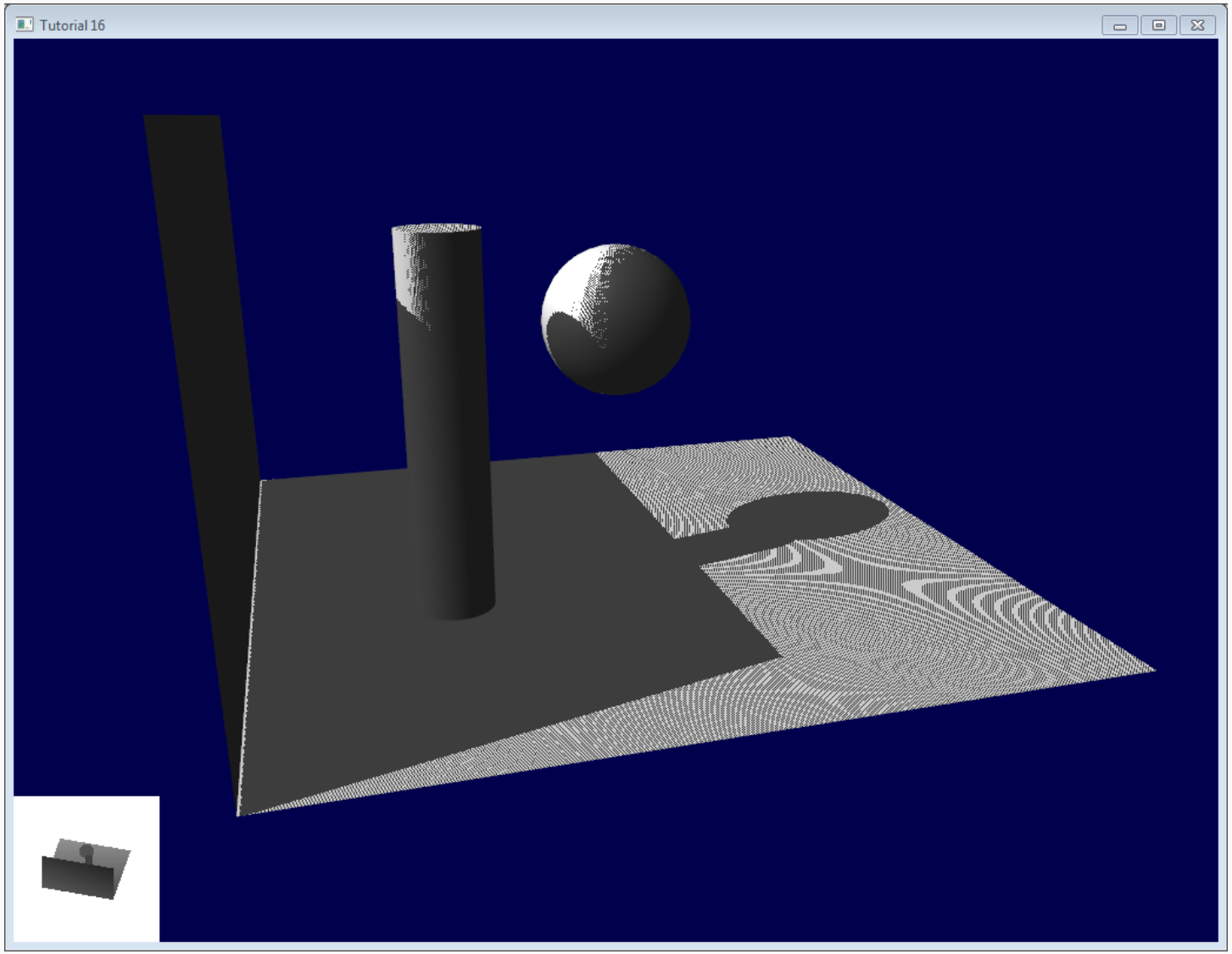
  var is_in_shadow = false;
  if (all(abs(shadow_coord.xy) < vec2f(1.0))) {
    let shadow_texcoord = vec2f(0.5)
      + shadow_coord.xy * vec2f(0.5, -0.5);

    is_in_shadow = textureSample(shadow_map, shadow_map_sampler,
      shadow_texcoord) < shadow_coord.z;
  }
}
```

Shadow mapping: буфер глубины



Shadow mapping: результат



Shadow mapping: проекция

- Как выбрать проекцию камеры (`shadow_projection`) для shadow map?

Shadow mapping: проекция

- Как выбрать проекцию камеры (`shadow_projection`) для shadow map?
- Удалённый источник освещает всю сцену с одного направления $\Longrightarrow$ удобно использовать ортографическую проекцию

Shadow mapping: проекция

- Как выбрать проекцию камеры (`shadow_projection`) для shadow map?
- Удалённый источник освещает всю сцену с одного направления $\Longrightarrow$ удобно использовать ортографическую проекцию
- Проекцию надо подобрать так, чтобы в видимую область попала вся сцена

Shadow mapping: проекция

- Как выбрать проекцию камеры (`shadow_projection`) для shadow map?
- Удалённый источник освещает всю сцену с одного направления $\Longrightarrow$ удобно использовать ортографическую проекцию
- Проекцию надо подобрать так, чтобы в видимую область попала вся сцена
- Можно спроецировать все объекты на плоскость, перпендикулярную направлению на свет (которое можно считать Z-осью этой камеры), взять любые два ортогональных вектора X и Y в этой плоскости, и найти bounding box сцены в координатах XYZ – это даст размеры и ориентацию области shadow map

Shadow mapping: проекция

- Точечный источник светит во все стороны $\Longrightarrow$ удобно использовать перспективную проекцию и cubemap-текстуру

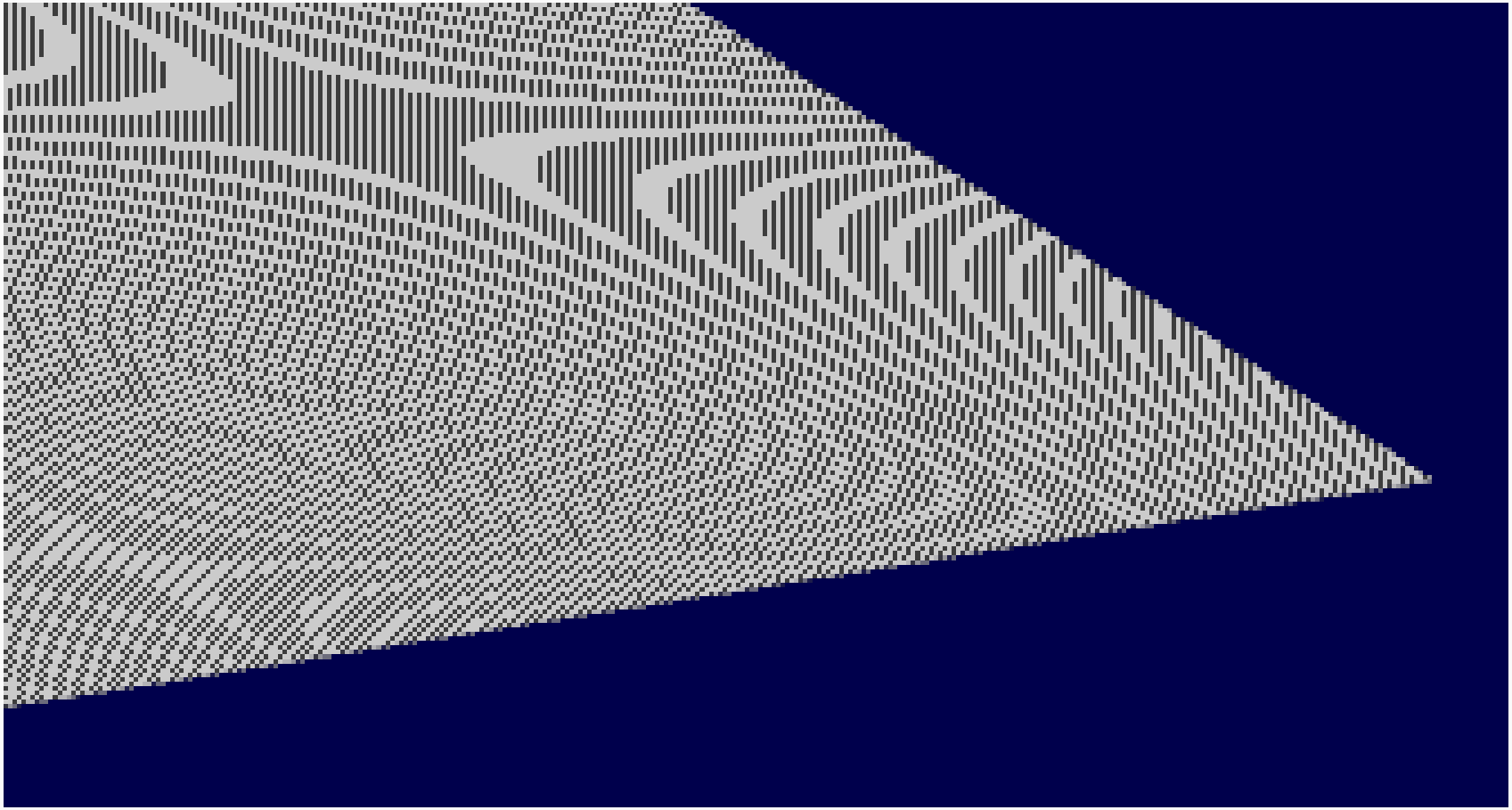
Shadow mapping: проекция

- Точечный источник светит во все стороны $\Longrightarrow$ удобно использовать перспективную проекцию и cubemap-текстуру
- 6 shadow map (по одной на каждую грань) с соответствующей перспективной проекцией

Shadow mapping: проекция

- Точечный источник светит во все стороны $\Longrightarrow$ удобно использовать перспективную проекцию и cubemap-текстуру
- 6 shadow map (по одной на каждую грань) с соответствующей перспективной проекцией
- Положение камеры совпадает с положением источника света, виртуальный ‘экран’ совпадает с отрисовываемой гранью shadow map

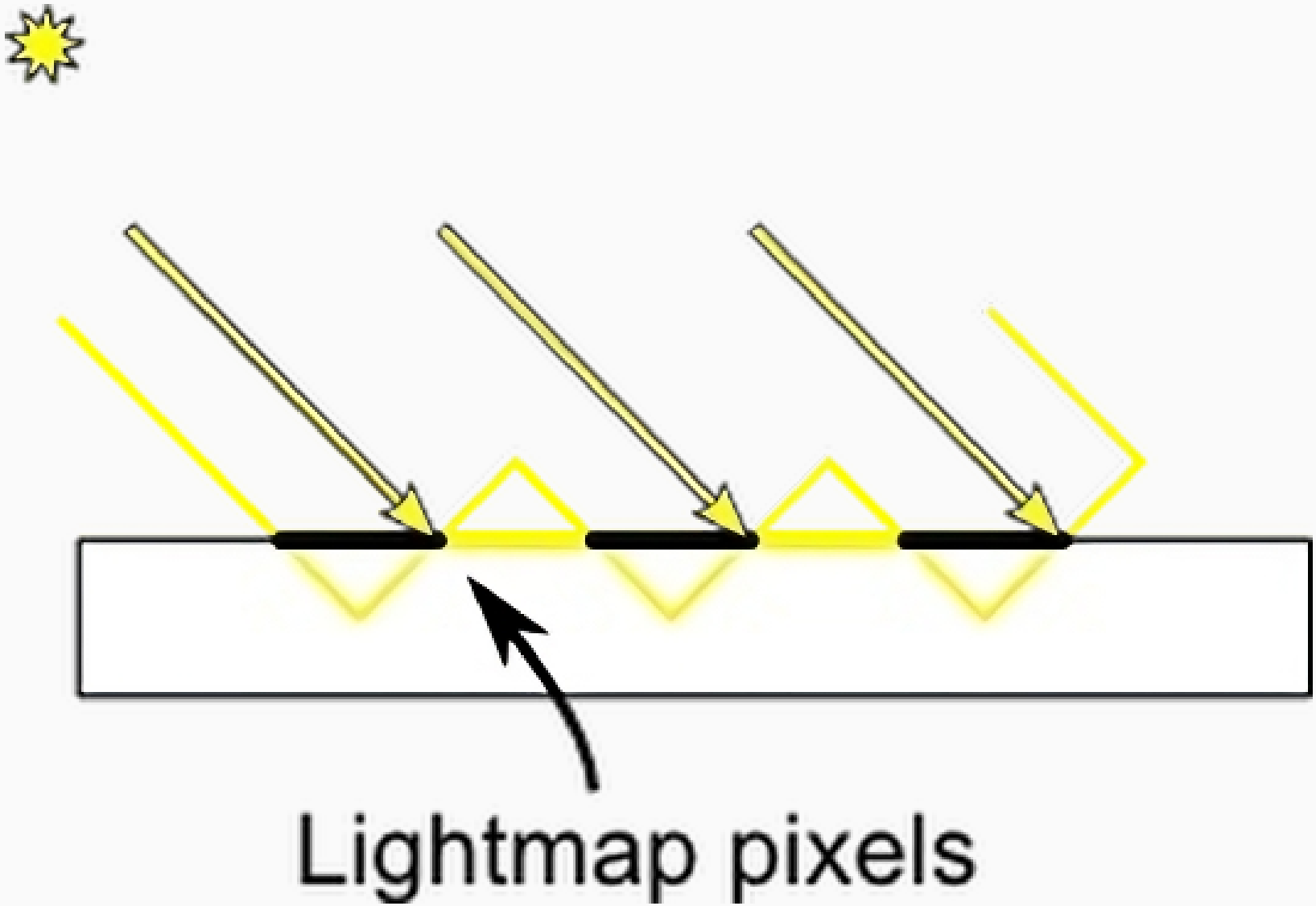
Shadow acne



Shadow acne

- Артефакт, возникающий из-за дискретности shadow map: если луч света не перпендикулярен плоскости объекта, рядом с центром пикселя shadow map будут точки с меньшей или большей глубиной

Shadow acne



Shadow acne

- Артефакт, возникающий из-за дискретности shadow map: если луч света не перпендикулярен плоскости объекта, рядом с центром пикселя shadow map будут точки с меньшей или большей глубиной

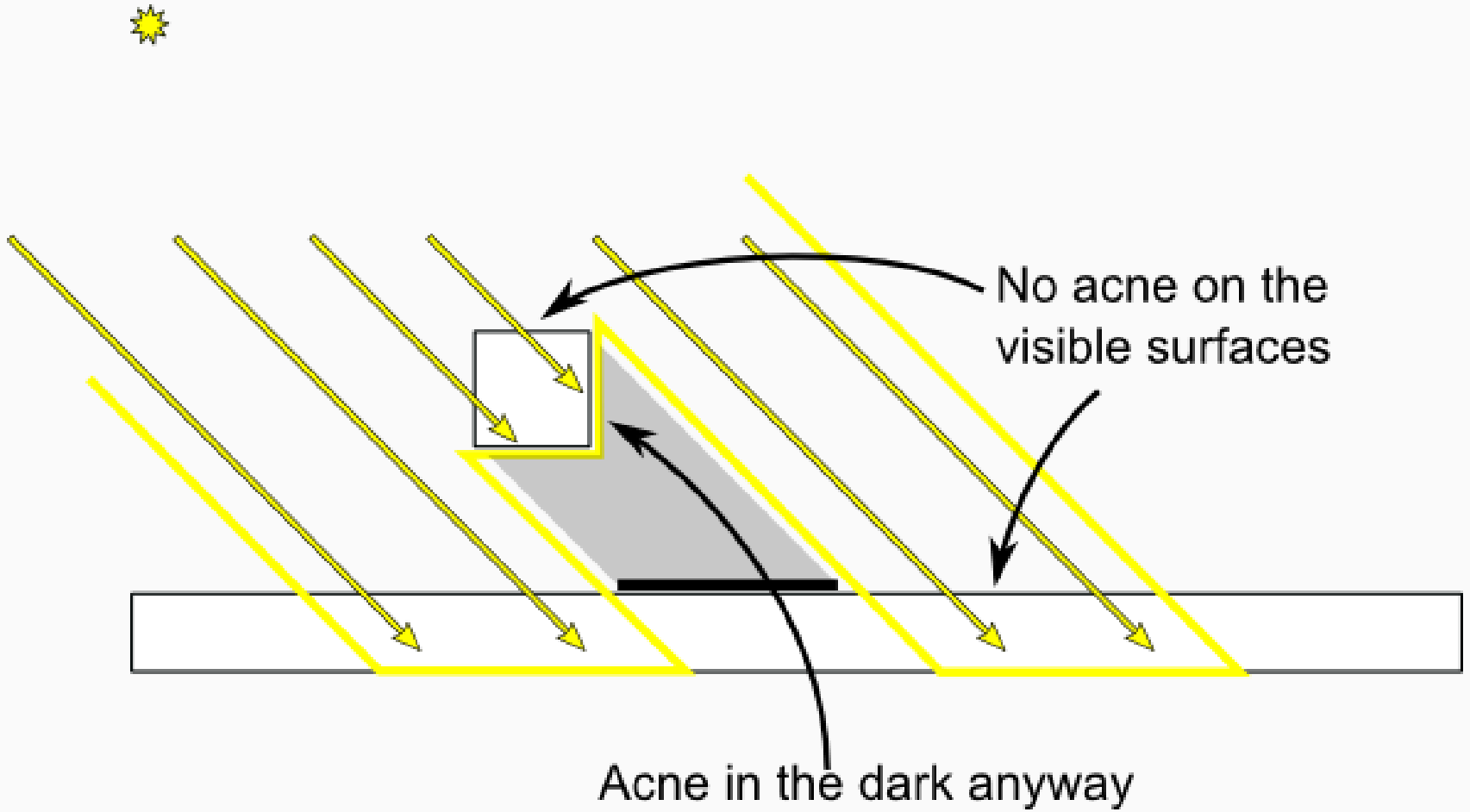
Shadow acne

- Артефакт, возникающий из-за дискретности shadow map: если луч света не перпендикулярен плоскости объекта, рядом с центром пикселя shadow map будут точки с меньшей или большей глубиной
- Первый способ исправить: добавить к значению, прочитанному из из shadow map, некую константу (shadow bias)
    - Часто эту константу умножают на тангенс угла падения света

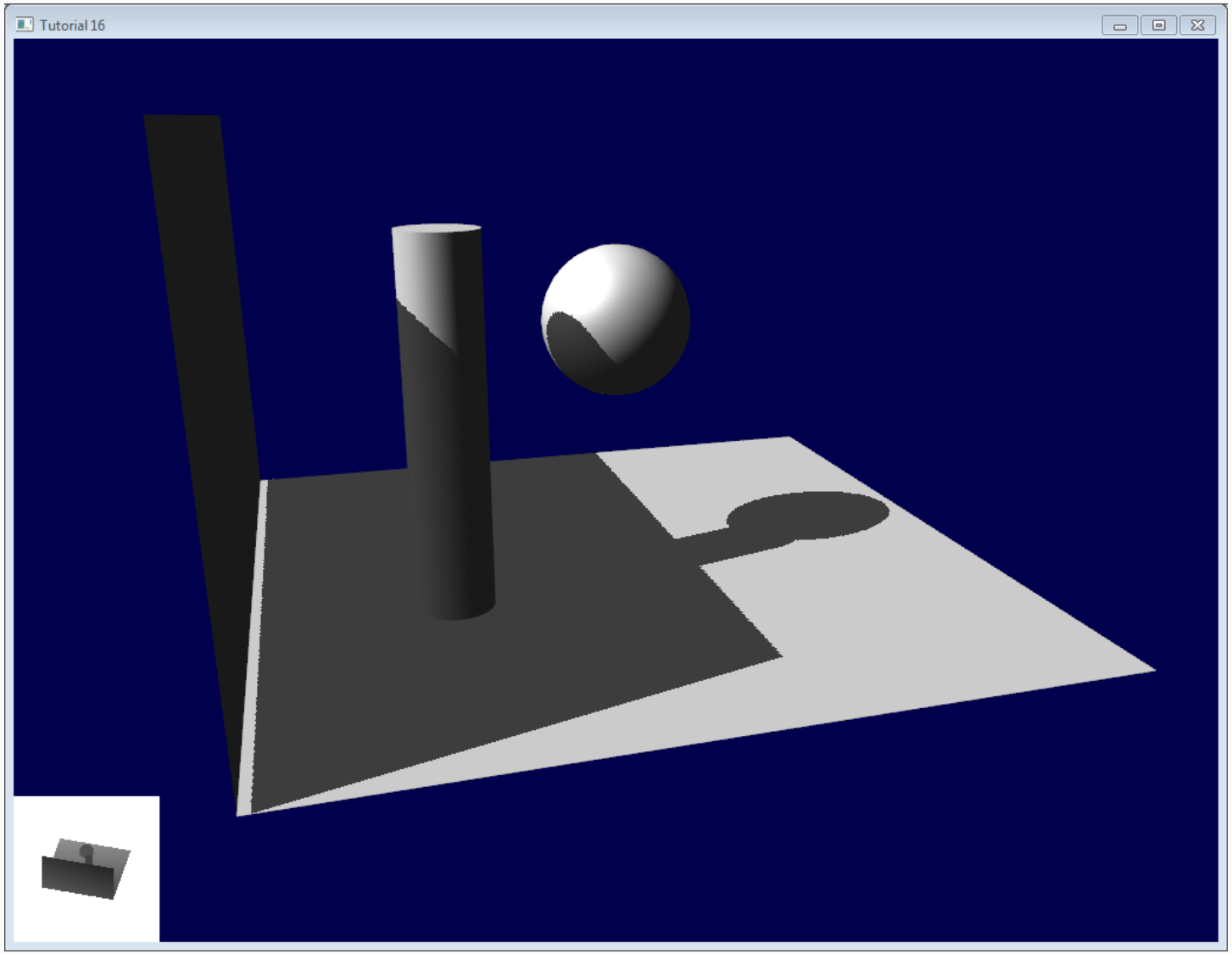
Shadow acne

- Артефакт, возникающий из-за дискретности shadow map: если луч света не перпендикулярен плоскости объекта, рядом с центром пикселя shadow map будут точки с меньшей или большей глубиной
- Первый способ исправить: добавить к значению, прочитанному из из shadow map, некую константу (shadow bias)
  - Часто эту константу умножают на тангенс угла падения света
- Второй способ исправить: рисовать только back-facing грани (т.е. обращённые против света) в shadow map
  - Теперь shadow acne возникнет на back-facing гранях, но мы и без shadow map знаем, что они в тени!
  - Работает только с правильной геометрией (двусторонней, без дырок, и т.п.)

Back-facing shadows



Peter panning



Peter panning

- Объекты кажутся приподнятыми над поверхностью из-за того, что их тень начинается не сразу под объектом

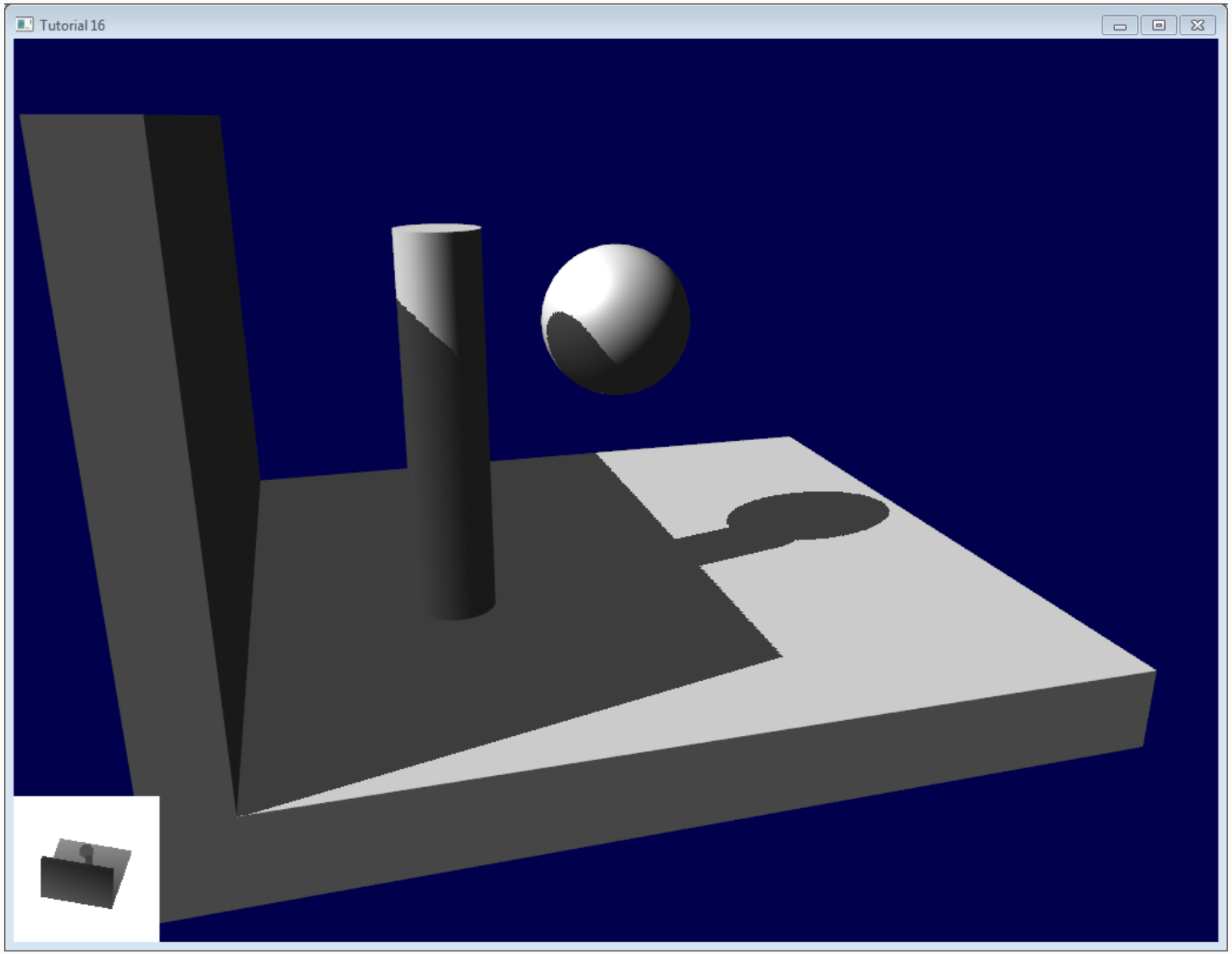
Peter panning

- Объекты кажутся приподнятыми над поверхностью из-за того, что их тень начинается не сразу под объектом
- Усиливается при использовании shadow bias

Peter panning

- Объекты кажутся приподнятыми над поверхностью из-за того, что их тень начинается не сразу под объектом
- Усиливается при использовании shadow bias
- Лучший способ исправить – не использовать слишком тонкую геометрию

Shadow mapping



Разрешение shadow map





Shadow mapping

- При низком разрешении shadow map хорошо видны ‘пиксели’ теней

Shadow mapping

- При низком разрешении shadow map хорошо видны ‘пиксели’ теней
- Чем выше разрешение, тем лучше жёсткая тень

Shadow mapping

- При низком разрешении shadow map хорошо видны ‘пиксели’ теней
- Чем выше разрешение, тем лучше жёсткая тень
  - Алгоритм silhouette mapping позволяет получить жёсткую тень без слишком большого разрешения

Shadow mapping

- При низком разрешении shadow map хорошо видны ‘пиксели’ теней
- Чем выше разрешение, тем лучше жёсткая тень
  - Алгоритм silhouette mapping позволяет получить жёсткую тень без слишком большого разрешения
- Можно слегка размыть тень, избавившись от ‘пикселей’ и получив аппроксимацию мягких теней

Shadow mapping

- При низком разрешении shadow map хорошо видны ‘пиксели’ теней
- Чем выше разрешение, тем лучше жёсткая тень
  - Алгоритм silhouette mapping позволяет получить жёсткую тень без слишком большого разрешения
- Можно слегка размыть тень, избавившись от ‘пикселей’ и получив аппроксимацию мягких теней
  - Размывать нужно результат сравнения глубины со значением в shadow map, а **не** саму shadow map!

Shadow mapping

- При низком разрешении shadow map хорошо видны ‘пиксели’ теней
- Чем выше разрешение, тем лучше жёсткая тень
  - Алгоритм silhouette mapping позволяет получить жёсткую тень без слишком большого разрешения
- Можно слегка размыть тень, избавившись от ‘пикселей’ и получив аппроксимацию мягких теней
  - Размывать нужно результат сравнения глубины со значением в shadow map, а **не** саму shadow map!
  - Этот приём известен как *percentage-closer filtering (PCF)*

Shadow mapping

- При низком разрешении shadow map хорошо видны ‘пиксели’ теней
- Чем выше разрешение, тем лучше жёсткая тень
  - Алгоритм silhouette mapping позволяет получить жёсткую тень без слишком большого разрешения
- Можно слегка размыть тень, избавившись от ‘пикселей’ и получив аппроксимацию мягких теней
  - Размывать нужно результат сравнения глубины со значением в shadow map, а **не** саму shadow map!
  - Этот приём известен как *percentage-closer filtering (PCF)*
  - В WebGPU есть встроенная реализация в виде *comparison sampling*

Shadow mapping

- При низком разрешении shadow map хорошо видны ‘пиксели’ теней
- Чем выше разрешение, тем лучше жёсткая тень
  - Алгоритм silhouette mapping позволяет получить жёсткую тень без слишком большого разрешения
- Можно слегка размыть тень, избавившись от ‘пикселей’ и получив аппроксимацию мягких теней
  - Размывать нужно результат сравнения глубины со значением в shadow map, а **не** саму shadow map!
  - Этот приём известен как *percentage-closer filtering (PCF)*
  - В WebGPU есть встроенная реализация в виде *comparison sampling*
- Более современные вариации shadow maps пишут в текстуру не глубину, а какие-нибудь значения, которые имеет смысл складывать и усреднять

Shadow mapping

- При низком разрешении shadow map хорошо видны ‘пиксели’ теней
- Чем выше разрешение, тем лучше жёсткая тень
  - Алгоритм silhouette mapping позволяет получить жёсткую тень без слишком большого разрешения
- Можно слегка размыть тень, избавившись от ‘пикселей’ и получив аппроксимацию мягких теней
  - Размывать нужно результат сравнения глубины со значением в shadow map, а **не** саму shadow map!
  - Этот приём известен как *percentage-closer filtering (PCF)*
  - В WebGPU есть встроенная реализация в виде *comparison sampling*
- Более современные вариации shadow maps пишут в текстуру не глубину, а какие-нибудь значения, которые имеет смысл складывать и усреднять
  - Позволяет использовать фильтрацию текстур, mipmaps, размытие shadow map

Shadow mapping

- При низком разрешении shadow map хорошо видны ‘пиксели’ теней
- Чем выше разрешение, тем лучше жёсткая тень
  - Алгоритм silhouette mapping позволяет получить жёсткую тень без слишком большого разрешения
- Можно слегка размыть тень, избавившись от ‘пикселей’ и получив аппроксимацию мягких теней
  - Размывать нужно результат сравнения глубины со значением в shadow map, а **не** саму shadow map!
  - Этот приём известен как *percentage-closer filtering (PCF)*
  - В WebGPU есть встроенная реализация в виде *comparison sampling*
- Более современные вариации shadow maps пишут в текстуру не глубину, а какие-нибудь значения, которые имеет смысл складывать и усреднять
  - Позволяет использовать фильтрацию текстур, mipmaps, размытие shadow map
  - Exponential shadow maps, variance shadow maps

Comparison sampling

- Нужно создать специальный sampler с настройкой `compare` – она будет управлять тем, какой тип сравнения будет происходить (`Less`, `Equal`, и т.п.)

Comparison sampling

- Нужно создать специальный sampler с настройкой `compare` – она будет управлять тем, какой тип сравнения будет происходить (`Less`, `Equal`, и т.п.)
- В bind group layout нужно указать

  `sampler_binding_type = Comparison`

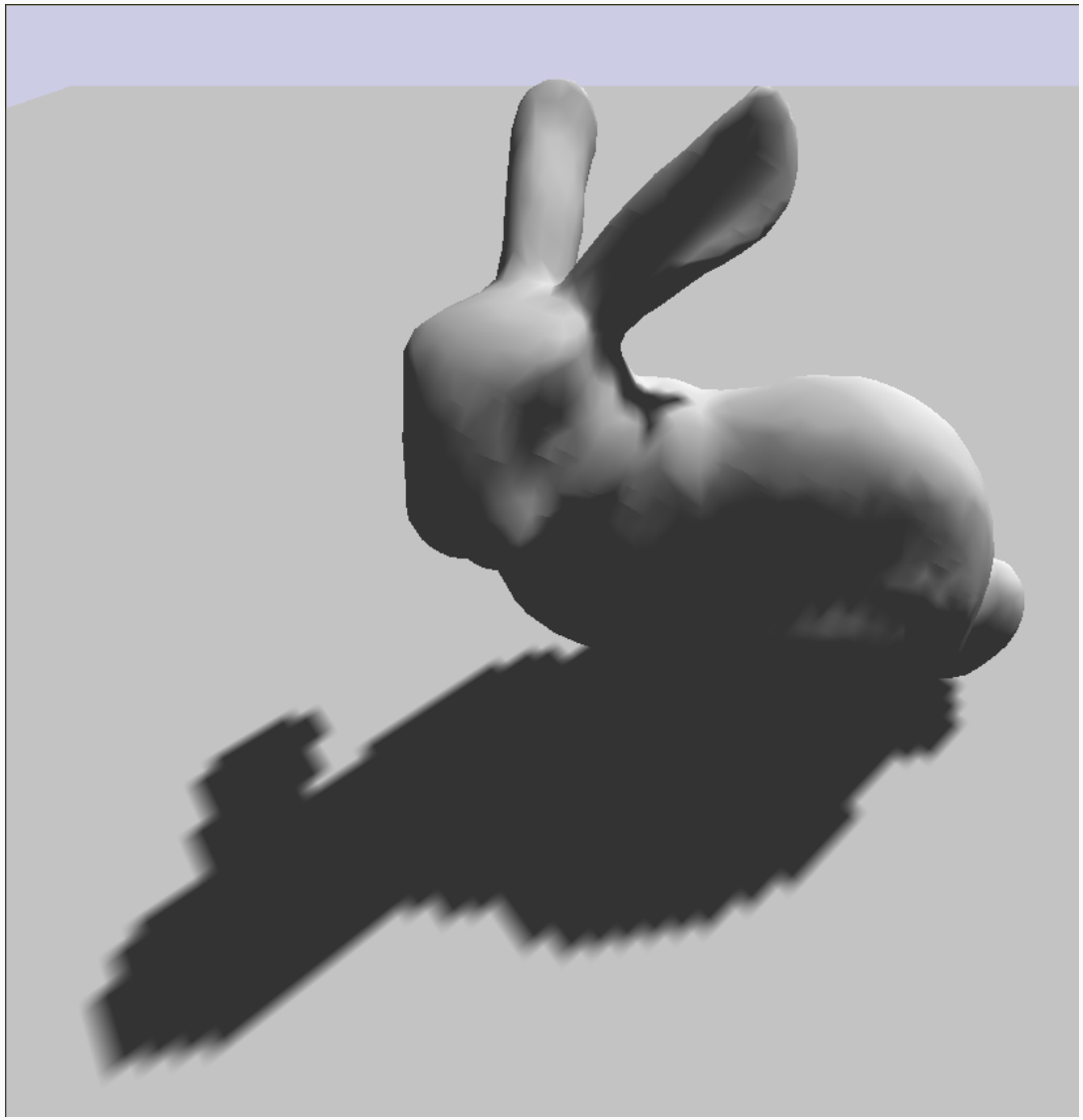
Comparison sampling

- Нужно создать специальный sampler с настройкой `compare` – она будет управлять тем, какой тип сравнения будет происходить (`Less`, `Equal`, и т.п.)
- В bind group layout нужно указать `sampler_binding_type = Comparison`
- В шейдере нужно объявить uniform-переменную типа `sampler_comparison`

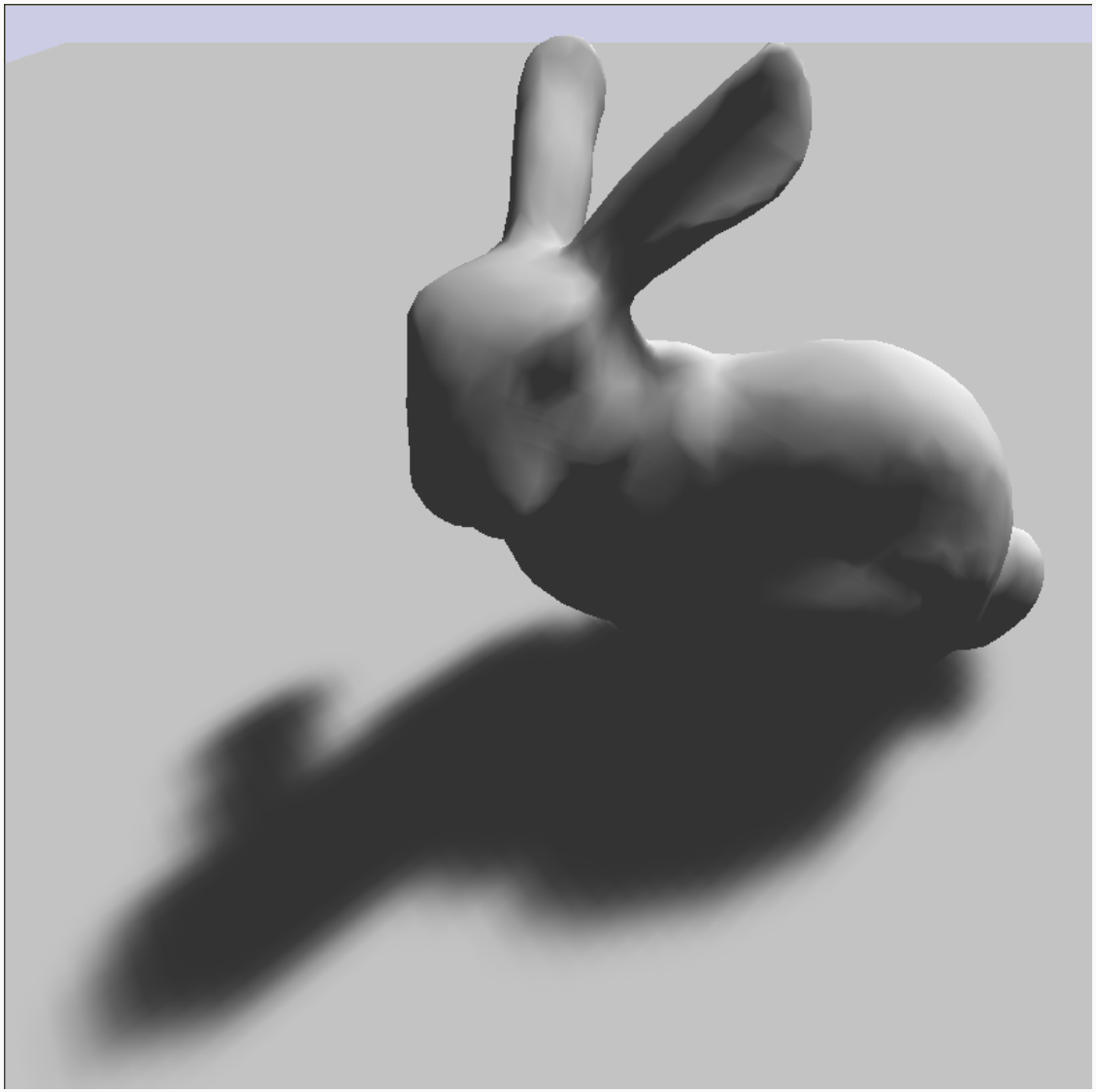
Comparison sampling

- Нужно создать специальный sampler с настройкой `compare` – она будет управлять тем, какой тип сравнения будет происходить (`Less`, `Equal`, и т.п.)
- В bind group layout нужно указать `sampler_binding_type = Comparison`
- В шейдере нужно объявить uniform-переменную типа `sampler_comparison`
- В шейдере используем функцию `textureSampleCompare()` – она сравнивает значения глубин соседних пикселей с переданной глубиной, и возвращает результат билинейной интерполяции результатов сравнения

Shadow sampler с линейной фильтрацией



Shadow sampler с линейной фильтрацией и 7x7 размытием по Гауссу



Shadow mapping: ссылки

- learnopengl.com/Advanced-Lighting/Shadows/Shadow-Mapping
- opengl-tutorial.org/intermediate-tutorials/tutorial-16-shadow-mapping
- ogldev.org/www/tutorial23/tutorial23.html
- GPU Gems 2, Chapter 17. Efficient Soft-Edged Shadows Using Pixel Shader Branching